Abstract

Time series data are widely used in financial market forecasting, meteorological monitoring, traffic flow analysis, and energy consumption prediction. With increasing application requirements, capturing distant temporal relationships in prolonged sequential data while preserving processing efficiency has emerged as a significant hurdle. Among traditional methods, Recurrent Neural Networks (RNNs) encounter gradient decay issues, limiting their ability to recognize patterns across extended sequences; Transformers, despite their global modeling capabilities, exhibit quadratic computational complexity with sequence length; standard State Space Models (SSMs) offer linear complexity advantages but fail to consider frequency domain information, limiting their ability to comprehensively model complex multi-scale sequence structures. The key challenge in this research is how to simultaneously capture both temporal and frequency domain features while maintaining computational efficiency, especially for multi-channel time series data.

To address this, we present a novel approach called Channel-aware Structured State Space Sequence Model (CS4) that maintains the linear computational advantages of the original SSM while achieving multi-channel time-frequency fusion modeling capabilities. The model features three key design elements: first, the introduction of instance normalization and channel reshuffling mechanisms to enhance representation capabilities for multi-channel inputs; second, the construction of a dual-branch architecture for time and frequency domains, employing improved Structured State Space Models with Diagonal parameterization (S4D) and Fast Fourier Transform combined with spectral attention mechanisms, effectively extracting dynamic dependencies and periodic features; third, the design of fusion strategies at multiple stages of the model to balance information sharing and independence between channels, significantly improving overall prediction performance.

Across seven public datasets including ETT, Electricity, Traffic and Weather, comparative analysis between CS4 and five baseline models demonstrates that CS4 outperforms competitors in 14 forecasting tasks spanning forecast horizons of {96, 192, 336, 720} time steps, achieving top-ranking outcomes in 11 experiments and second-place results in 3 instances, with an average MSE reduction of 17.4%. In terms of computational efficiency, CS4 reduces training time by 18.8% compared to PatchTST and consumes only 81.5% of the GPU memory required by TimesNet. Particularly in long sequence prediction tasks, its dual-branch architecture demonstrates advantages in precisely modeling complex patterns while maintaining moderate resource consumption. This research validates the theoretical value of

time-frequency fusion strategies based on structured state space models for multi-channel long sequence modeling, demonstrating their extensive potential as an alternative pathway to traditional recurrent structures and attention mechanisms.

Keywords: Deep Learning, Long-Range Dependencies, Structured State Space Models, Time-frequency Domain Fusion, Multi-channel Time Series Forecasting

Contents

Abstract 3

Acknowledgements 5

1 Introduction 10

1.1 Research Background 10

1.2 Literature Review 11

1.3 Main Contributions 14

1.4 Main Structure 15

2 Preliminary 17

2.1 State Space Models 17

2.1.1 Continuous-time State Space Model 18

2.1.2 Discrete-time State Space Model 18

2.2 Deep Structured State Space Models 20

2.2.1 HiPPO Structure 20

2.2.2 Structured State Space Models and Variants 22

2.3 Attention Mechanism 23

2.4 Gating Mechanism 24

2.5 Fast Fourier Transform 25

3 Methodology 26

3.1 Model Architecture 26

3.2 CS4 Encoder Components 27

3.2.1 Temporal Feature Extraction 28

3.2.2 Spectral Feature Extraction 30

3.2.3 Feature Fusion 31

3.3 Model Effectiveness Analysis 31

3.3.1 Accuracy Analysis 31

3.3.2 Computational Complexity Analysis 32

3.3.3 Parameter Complexity Analysis 33
3.4 Model Generation Process 34
4 Experiements 37
4.1 Experimental Setup 37
4.2 Comparative Model Setup 38
4.3 Comparative Model Setup 39
4.3.1 Parameter Sensitivity Analysis 39
4.3.2 Comparative Analysis with Baseline Models 41
4.3.3 Computational Efficiency Analysis 46
5 Conclusion 48
References 50

List of Figures

Figure 1 Three representations of structured state space models point [12].....17

Figure 2 Recursive representation of cyclic units of SSMs layers in the first *h* dimension 19

Figure 3 Overall architecture of Channel-aware Structured State Space Sequence model 26

Figure 4 Overall architecture of the CS4 Encoder(time-branch and freq-branch) 28

Figure 5 Computational framework of the S4D module and its FFT-based acceleration 29

Figure 6 Heat map of MSE of CS4 model with different layer configurations on ETTh1, ETTm1 and Traffic datasets.....39

Figure 7 MAE performance trends of CS4 and baseline models across different prediction lengths.....43

Figure 8 Comparison of prediction performance between CS4(left) and Rlinear(right) on the ETTh2 dataset across different prediction lengths.....44

Figure 9 Comparison of prediction performance between CS4(left) and PatchTST(right) on the ETTm2 dataset across different prediction lengths.....45

Figure 10 Bubble chart of computational efficiency versus prediction error for each model on the ETTh2 dataset. 46

Figure 11 Modifications of the four chosen baselines. We retain the original architecture unchanged but apply Channel Mixup during training 55

Figure 12 Multi-channel time series prediction results of CS4 model under ETTh1 dataset..65

Figure 13 Multi-channel time series prediction results of CS4 model under ETTh2 dataset..65

Figure 14 Multi-channel time series prediction results of CS4 model under ETTm1 dataset.66

Figure 15 Multi-channel time series prediction results of CS4 model under ETTm2 dataset.66

Figure 16 Multi-channel time series prediction results of CS4 model under Weather dataset67

Figure 17 Prediction cases for ETTh1 under CS4 67

Figure 18 Prediction cases for ETTm1 under CS4. 67

Figure 19 Prediction cases for Electricity under CS4. 67

Figure 20 Prediction cases for Weather and Traffic under CS4. 68

List of Tables

Table 1 Information about seven datasets used for the experiment 37

Table 2 Impact of increasing CS4 model layers from 2 to 4 on prediction performance (percentage change in MSE relative to the layer=2 baseline) 40

Table 3 Performance comparison of CS4 and baseline models across five datasets using MSE and MAE metrics 42

Table 4 Computational efficiency comparison on the ETTh2 dataset (pred_len=96, layer=2, d_model=256) 46

Table 5 Impact of prediction length on the predictive performance of the CS4 model (% MSE growth rate with pred_len=96) 56

Table 6 Impact of the number of layers on the predictive performance of the CS4 model (percent change in MSE with layer=2) 57

Table 7 Impact of model width on CS4 model prediction performance (percent change in MSE from 128 to 256 for d_model). 57

Table 8 Complete experimental results of the CS4 model on the ETTh1 dataset 58

Table 9 Complete experimental results of the CS4 model on the ETTm1 dataset 59

Table 10 Complete experimental results of the CS4 model on the ETTh1 dataset 60

Table 11 Complete experimental results of the CS4 model on the ETTm2 dataset 61

Table 12 Complete experimental results of the CS4 model on the Electricity dataset 61

Table 13 Complete experimental results of the CS4 model on the Weather dataset 62

Table 14 Complete experimental results of the CS4 model on the Traffic dataset 63

Table 15 Performance of CS4 vs. baseline model on five datasets 64

Table 16 Computational efficiency of CS4 vs. baseline model on ETTh2 dataset 64

1 Introduction

1.1 Research Background

Time series data, as a form of data continuously sampled along the time dimension, are widely encountered across diverse fields such as financial market forecasting, meteorological monitoring, traffic flow analysis [1], and energy consumption prediction. Accurate modeling and forecasting of long time series data has become one of the core challenges in the field of data science. As IoT technologies and sensing infrastructures continue to evolve at an accelerating pace, the dimensionality and complexity of time series data have grown exponentially, rendering traditional sequence modeling methods inadequate to handle the increasingly large-scale and complex data.

The long-range dependency issue represents a fundamental challenge in time series modeling. In real-world scenarios, the current state is often influenced by historical information spanning extensive time periods, capturing these long-term dependencies is crucial for improving prediction accuracy. Earlier sequence modeling approaches including RNNs and their evolutionary developments [2-3], specifically LSTM and GRU architectures, encounter gradient decay and amplification issues when processing extended data streams [4], limiting their capacity to accurately represent extended sequential relationships [5].

Recently, sequence modeling has seen remarkable advances through the implementation of Transformer designs utilizing self-attention approaches [6-9]. Transformer establishes direct associations between any two time points in a sequence through self-attention, theoretically capable of capturing dependencies of arbitrary length [6]. Nevertheless, traditional Transformer models suffer from quadratic $O(L^2)$ computational complexity as sequence length increases, resulting in prohibitively high computational costs and memory consumption when processing long sequence, constraining its effectiveness when forecasting patterns across lengthy temporal horizons [10].

To address these issues, state space models (SSMs) have increasingly attracted attention [11]. Drawing upon state space representations from classical control theory, SSMs discretize continuous dynamical systems, achieving linear computational complexity $O(L)$ [12]. The Structured State Space Sequence Model (S4) marks a notable breakthrough in the development of State Space Models, combining the Hierarchical Prior Preservation Operator (HiPPO) theory to enhance its modeling capabilities further [13-15]. By employing a specialized parameterization, HiPPO enables SSMs to preserve historical information more effectively, thus better capturing long-term dependencies [15]. Compared with traditional RNNs, SSMs

handle long sequence inputs more stably without being affected by gradient vanishing problems; compared to Transformers, SSMs significantly reduce computational complexity while maintaining the capability to model long-range dependencies.

However, existing S4 models and their variants primarily focus on extracting temporal domain features, with relatively limited utilization of frequency domain information [11]. Time series data typically encompass rich features in both the time and frequency domains: temporal features reflect immediate changes and trends, while frequency domain features reveal periodic patterns and hidden regularities [14]. Limiting analysis to just one domain proves insufficient for fully understanding sequence attributes, especially when dealing with temporal data containing cyclic patterns or features at multiple scales.

Therefore, this study builds upon the S4 model to explore the simultaneous utilization of temporal and frequency domain information in sequence modeling. Considering the S4 model's natural strengths in modeling extended sequential relationships, this research examines approaches for merging frequency-based analytical techniques with channel integration strategies to improve modeling of complex temporal patterns. This work aims to construct a state space framework that successfully incorporates channel-specific sequence properties across both time and frequency perspectives, delivering a more thorough and computationally practical approach for extended chronological predictions.

1.2 Literature Review

Recurrent Neural Networks (RNNs) and their variants have historically served as core architectural components for sequential temporal forecasting problems [2-3]. The Long Short-Term Memory (LSTM) framework initially developed by Hochreiter and Schmidhuber in 1997, introducing gating mechanisms and memory cells, partially alleviated the gradient vanishing problem inherent in traditional RNNs [3]. Cho et al. (2014) contributed the Gated Recurrent Unit framework, which preserved LSTM's effectiveness while reducing architectural complexity [16]. Nevertheless, as noted by Bengio et al. (1994), these recurrent architectures still suffer from gradient instability issues when dealing with long sequences, thus struggling to recognize patterns spanning significant sequential intervals [17]. Addressing this constraint, Bai et al. (2018) developed the Temporal Convolutional Network (TCN), applying Convolutional Neural Networks (CNNs) to sequence modeling and expanding the receptive field through causal and dilated convolutions [18]. Despite these improvements, due to their architecture focusing on neighboring regions, CNNs struggle to detect relationships across

distant portions of the input, especially for time series exhibiting complex, long-term correlations.

Vaswani et al. (2017) introduced the Transformer architecture, which achieves direct interactions between elements at arbitrary positions in sequences via the self-attention mechanism, marking a breakthrough across various sequence modeling tasks [6-8]. Within the field of sequential data prediction over time, models such as Informer by Li et al. (2019) [19] and Autoformer by Zhou et al. (2021) [20], both built upon the Transformer framework, underwent refinements to better handle the distinctive properties of sequential temporal information. However, as demonstrated by Tay et al. (2020) in the Long Range Arena (LRA) benchmark [21], the standard Transformer's self-attention mechanism incurs quadratic computational complexity $O(L^2)$, presenting significant computational and memory bottlenecks for long sequences. To reduce complexity, several improvements have been proposed: Child et al. (2019) designed the Sparse Transformer, reducing computations through sparse attention patterns [22]; Katharopoulos et al. (2020) proposed the Linear Transformer, converting attention calculations to linear complexity [23]. Although these optimizations enhance computational efficiency, they often sacrifice the ability to capture long-range dependencies effectively.

As a novel paradigm for handling long sequences, State Space Models (SSMs) have recently attracted considerable attention. Gu et al. (2021) proposed Linear State Space Layers (LSSL) into the deep learning field, offering a fresh approach for modeling long sequences [24]. Compared with the Transformer's quadratic complexity ($O(L^2)$), SSM achieves linear computational complexity ($O(L)$), greatly enhancing efficiency in handling long sequences. The fundamental idea behind SSMs is discretizing continuous dynamical systems for sequence modeling, representing system dynamics via state transition matrices, thus naturally modeling the evolution process of time series data. As discussed by Gu et al. (2022), SSM maps input sequences to hidden states and subsequently to outputs, forming an end-to-end trainable system whose computation can be efficiently implemented via convolution [11].

The emergence of HiPPO (Hierarchical Prior Preservation Operator) framework has considerably enhanced the capability of State Space Models to capture correlations across extended temporal sequences. The HiPPO theoretical foundation was established by the research team of Gu and colleagues in 2020, constructing a mathematical framework that efficiently preserves historical information, enabling states to maintain long-term memory through a specialized parameterization approach [15]. Building upon HiPPO theory, the research group led by Gu in 2022 introduced S4, an innovative approach to sequential modeling

based on structured state spaces [11], integrating HiPPO's parameterization method with state space modeling, additionally decreasing processing requirements from $O(L^2)$ to the more efficient O(Llog L) using Fast Fourier Transform (FFT). The S4 architecture demonstrated exceptional results on various extended-sequence challenges in Tay and colleagues' (2020) Long Range Arena (LRA) evaluation framework, highlighting its effectiveness at modeling distant temporal connections [25].

To further optimize the structure and performance of the S4 model, researchers have proposed various variants. Gu et al. (2022) developed S4D (S4-Diagonal), which simplifies computation and minimizes the quantity of model parameters through diagonalization of the state transition matrix while maintaining performance comparable to the original S4 [26]. Gupta et al. (2022) proposed the Diagonal State Space (DSS) model, further optimizing the structure of S4D by employing simpler parameterization methods, thereby reducing implementation complexity and enhancing training stability [27]. Recently, Gu and Dao (2023) introduced the Mamba model, employing an adaptive memory framework that modifies its retention capabilities based on contextual input patterns, creating fresh avenues for sequence-based computational approache [28].

In addition to purely temporal domain sequence modeling, methods combining temporal and frequency domain analyses have also been extensively studied. Wang et al. (2022) introduced FEDformer, incorporating Fourier transforms into the Transformer architecture to extract periodic patterns from time series through frequency-domain decomposition [29]. Wu et al. (2023) developed the novel TimesNet architecture, which converts sequential temporal data into 2D matrices to extract multi-level periodic patterns through frequency-time analysis methods [30]. Liu et al. (2022) proposed SCINet, employing wavelet transforms for multi-scale frequency domain analysis to effectively extract hierarchical features from time series data [31]. Nie et al. (2023) introduced PatchTST, an approach that segments temporal sequences into localized chunks and implements attention-based processing, achieving superior feature extraction from time-based data while retaining computational performance advantages [32]. These studies indicate that integrating temporal and frequency domain analyses can more comprehensively capture the complex characteristics of time series.

In recent years, research on hybrid models has demonstrated significant potential. Liu et al. (2024) introduced CMamba, which unites the strengths of convolutional architectures for nearby pattern recognition with state space formulations for distant relationship modeling, achieving remarkable outcomes across multiple time series forecasting tasks [33]. Zhang et al. (2023) developed CTRW-SSM, integrating continuous-time random walk theory with state

space modeling to effectively handle irregularly sampled time series data [34]. Pan et al. (2023) designed STID-S4, merging spatiotemporal interaction dynamics modules with the S4 model to enhance modeling capabilities for multivariate time series [35]. Mehta et al. (2023) proposed the Hyena architecture, combining long convolutions with SSM to capture ultra-long-range dependencies efficiently while maintaining linear computational complexity [36]. These hybrid frameworks suggest that integrating the strengths of diverse models provides a more comprehensive solution to address the challenges inherent in time series modeling.

1.3 Main Contributions

In the analysis of existing research presented in Section 1.2, we deeply investigated the developments and limitations of current time series prediction methods. Despite significant advancements in modeling long sequences using SSMs, two major challenges remain. First, existing SSM approaches predominantly focus on temporal domain feature extraction, with insufficient utilization of frequency domain information, although time series data often encompass important features from both temporal and frequency domains simultaneously [19,31]. Second, strategies for modeling inter-channel interactions and feature fusion in multi-channel time series need further optimization [31-37]. With the rapid advancement of IoT and sensor technologies, there is an increasing demand for analyzing multi-channel long time series data, thus necessitating the development of efficient models capable of concurrently exploiting temporal and frequency domain information and effectively handling multi-channel features. Accordingly, this study explores a novel approach aimed at preserving the advantages of SSMs in modeling long-range dependencies while effectively integrating temporal and frequency domain features to enhance the predictive performance for multi-channel time series [38].

The primary contributions and works of this study are summarized as follows:

1. Firstly, we introduce a multi-channel temporal-frequency domain fusion SSM architecture named Channel-aware Structured State Space Sequence model (CS4). This design expands multi-channel analytical capabilities yet retains the computational advantages of linear complexity found in the standard S4 model. The CS4 model utilizes instance normalization and channel rearrangement techniques to handle multi-channel inputs, utilizing segmented encoding techniques to convert temporal sequences into formats optimized for neural network processing, laying a solid foundation for subsequent temporal-frequency domain modeling.
2. Furthermore, this study designs a dual-branch modeling mechanism from both temporal and frequency domains to effectively capture sequence characteristics. The temporal

branch employs an improved S4D module combined with a gating mechanism to model the dynamic temporal properties of sequences, effectively capturing long-range temporal dependencies. The component focused on frequency analysis converts temporal data into spectral representation via FFT algorithms, and incorporates a spectral attention mechanism to extract periodic patterns. This dual-branch architecture comprehensively captures the multi-dimensional characteristics of time series data, providing richer information for precise predictions.

3. Finally, this study develops an efficient multi-channel feature extraction and fusion strategy, achieving inter-channel feature interactions and fusion across various stages of the model architecture. At the patch embedding stage, the model initially maintains channel independence for feature extraction; within the CS4 encoder, channel-wise information sharing is realized through rearrangement operations; at the output stage, specially designed reshaping and flattening operations integrate features across channels to generate final predictions. This multi-level fusion strategy effectively balances channel independence and correlation, significantly improving the overall prediction performance for multi-channel time series.

This model not only theoretically proposes innovative methods but also empirically validates their effectiveness and practicality through extensive experimentation. The evaluation portion presents thorough testing using seven recognized public datasets—ETT, Electricity, Weather, Traffic among them—benchmarking the CS4 technique against contemporary approaches like iTransformer, PatchTST, TimesNet, RLinear, and LSTM. Model performance is assessed across multiple dimensions, including prediction accuracy, inference time, and memory consumption. The results indicate that the CS4 model achieves superior performance on most datasets and forecasting horizons. Particularly, in long-sequence forecasting tasks, the temporal-frequency domain fusion architecture of CS4 demonstrates clear advantages, validating our approach's strength for extended multi-dimensional sequential forecasting.

1.4 Main Structure

This paper consists of five chapters, systematically elaborating on research into multi-channel temporal-frequency domain fusion methods based on Structured State Space Models.

Chapter 1 provides an introductory overview, outlining the research context and practical importance of temporal sequence forecasting, while comprehensively examining the technological progression from RNN architectures to State Space modeling approaches. It further elaborates on the research motivations and major contributions of the proposed

Channel-aware Structured State Space Sequence model (CS4), laying the foundation for the subsequent chapters.

Chapter 2 covers related technologies and theoretical foundations. It systematically introduces the mathematical formulation of state space models along with their recursive and convolutional implementations. This study accentuates the HiPPO structure's ability to identify remote sequential patterns, paired with the improved architectural elements of the S4D implementation, followed by brief introductions to foundational theories such as the attention mechanism, gating units, and Fourier transform.

Chapter 3 outlines the technical specifications and operational mechanisms of the CS4 approach. It outlines the overall architecture and core components of the model, combining state-based temporal architecture in the time processing section and attention mechanisms for spectral analysis in the frequency component. Through theoretical analysis and pseudocode, the study verifies the model's practical benefits while documenting its implementation steps, establishing a straightforward technical reference

Chapter 4 presents experimental validation, systematically gauging the CS4 framework's capabilities through multiple experimental trials using seven accessible benchmark datasets. The comparative assessment against current top-performing frameworks across various metrics reveals our approach's superior capabilities in handling extended multi-channel forecasting challenges.

Chapter 5 provides conclusions and future perspectives. It summarizes the research outcomes and innovative aspects of this thesis while highlighting its current limitations. Based on findings and reflections from the research process, potential directions for future research are proposed, offering insights for the sustained development of this field.

2 Preliminary

This chapter provides a detailed introduction to the fundamental theories and techniques related to this thesis, including state space models, structured state space models and their variants, attention modules, gating units, as well as the basics of linear time-invariant systems and Fourier transforms.

2.1 State Space Models

Originating from control theory and first proposed by Kalman in the 1960s, the state space approach represents a traditional mathematical framework for modeling dynamical processes [39]. As a representation method for continuous-time dynamic systems, SSMs establish mappings between internal system states and external observations through a concise yet powerful set of mathematical equations, effectively capturing the temporal characteristics of system evolution. With the advancement of deep learning, SSMs have been incorporated into neural network architectures, establishing structured state space methodologies as a distinct conceptual foundation for processing extended sequential patterns.

SSMs are based on continuous-time dynamical system theory and can theoretically handle input sequences of infinite length while maintaining linear computational complexity, making them particularly important for long-sequence modeling tasks. The core idea of an SSM is to simplify complex sequence dynamics into two distinct processes: state transitions and observations. By employing carefully designed parameter matrices, an SSM efficiently encodes information across varying time scales. As illustrated in figure 1, structured state space models have three equivalent representations: continuous-time representation, recursive representation, and convolutional representation [11-13].

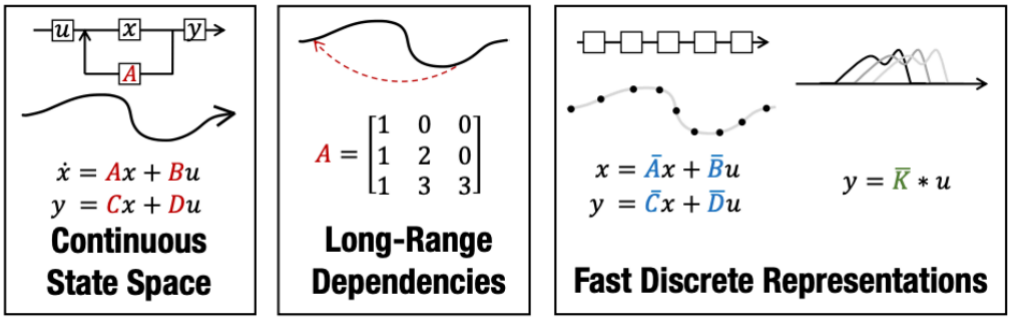


Figure 1. Three representations of structured state space models point [11].

2.1.1 Continuous-time State Space Model

The continuous-time variant of state space frameworks characterizes evolving system conditions via differential equations [11]. Equation (1) specifies the continuous-time state space framework, transforming a single-dimensional input $u(t)$ into an N-dimensional internal representation before converting back to a one-dimensional output $y(t)$.

$$\begin{aligned} x'(t) &= \boldsymbol{A}x(t) + \boldsymbol{B}u(t) \\ y(t) &= \boldsymbol{C}x(t) + \boldsymbol{D}u(t) \end{aligned} \tag{1}$$

Here, $x(t) \in \mathbb{R}^N$ denotes the hidden state vector of the system at time t, with N being the state dimension; $u(t) \in \mathbb{R}$ represents the one-dimensional input signal, and $y(t) \in \mathbb{R}$ is the one-dimensional output signal. $\boldsymbol{A} \in \mathbb{R}^{N\times N}$ is the state transition matrix, governing the dynamics of state evolution; $\boldsymbol{B} \in \mathbb{R}^{N\times 1}$ is the input matrix, determining how the input signal influences the states; $\boldsymbol{C} \in \mathbb{R}^{1\times N}$ is the output matrix mapping internal states to observable outputs, and $\boldsymbol{D} \in \mathbb{R}$ is the direct transfer matrix, representing the direct influence of the input on the output.

By formulating SSMs as deep sequential models in a black-box manner, temporal data fluctuations can be efficiently identified and processed [11,14]. In neural network applications, the key parameters $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}, and\ \boldsymbol{D}$ of the model are learned and optimized via gradient descent, allowing for customization to particular sequence analysis challenges. To simplify the model and focus on the primary discussion, this thesis chooses to omit the parameter $\boldsymbol{D}$, or equivalently assumes $\boldsymbol{D} = 0$ [11]. This simplification is justified since the term $\boldsymbol{D}u(t)$ can be interpreted as a type of skip connection within the model, and similar functionality can be implemented by other architectural designs in practical applications, thus simplifying computations without compromising model expressiveness.

2.1.2 Discrete-time State Space Model

In practical applications, we primarily deal with discrete time series data rather than continuous functions; consequently, transforming these continuous-time frameworks into discrete representations becomes essential [11]. Given a discrete input sequence $\{u_0, u_1, \dots, u_L\}$, we can regard it as samples from the continuous signal $u(t)$, such that $u_k = u(k\Delta)$, where the step size Δ represents the sampling interval and indicates the temporal resolution of the input. The discrete-time state space model can be implemented primarily in two ways: recursive implementation and convolutional implementation. Recursive implementation directly updates states and outputs according to the state equations at each timestep, akin to the computation method of Recurrent Neural Networks (RNNs) [2]. Alternatively, methods based on

convolution derive the impulse-based system behavior, expressing outputs as convolutions between inputs and the impulse response.

Recursive implementation adheres to the definition of state space equations, calculating states and outputs step by step. A commonly used discretization method is Zero-Order Hold (ZOH) [14]; however, in practical scenarios, the Bilinear Transform is widely adopted due to its superior numerical properties. When the parameter $\boldsymbol{D}$ is omitted, the discretized state space model can be represented by Equation (2):

$$\begin{aligned} x_k &= \bar{\boldsymbol{A}} x_{k-1} + \bar{\boldsymbol{B}} u_k \\ y_k &= \bar{\boldsymbol{C}} x_k \end{aligned} \tag{2}$$

The bilinear discretization process illustrated by Equation (3):

$$\begin{aligned} \bar{\boldsymbol{A}} &= \left(\boldsymbol{I} - \frac{\Delta}{2} \cdot \boldsymbol{A}\right)^{-1} \left(\boldsymbol{I} + \frac{\Delta}{2} \cdot \boldsymbol{A}\right) \\ \bar{\boldsymbol{B}} &= \left(\boldsymbol{I} - \frac{\Delta}{2} \cdot \boldsymbol{A}\right)^{-1} \Delta \cdot \boldsymbol{B} \\ \bar{\boldsymbol{C}} &= \boldsymbol{C} \end{aligned} \tag{3}$$

As illustrated in figure 2, this discretization approach transforms the continuous temporal framework into a function that relates discrete sequential inputs $\{u_k\}$ to corresponding discrete sequential outputs $\{y_k\}$, preserving the stability characteristics of the original system [40]. Particularly, as the step size Δ approaches zero, the discrete system behavior approaches that of the continuous system.

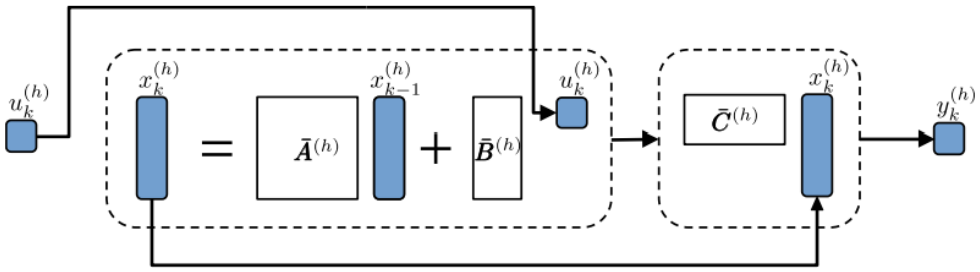


Figure 2. Recursive representation of cyclic units of SSMs layers in the first *h* dimension .

Similar to the limitations of RNNs, the recursively represented state space model struggles with parallelization, limiting its utilization of modern training hardware. Despite their recursive nature, LTI models allow for equivalent expression through discrete convolution mechanisms.

In general, the initial state $x_{-1} = 0$ of the SSMs can be learned. The Equation (2) can be obtained explicitly:

$$\begin{aligned} x_0 &= \overline{\boldsymbol{B}} u_0 \\ x_1 &= \overline{\boldsymbol{A}}\overline{\boldsymbol{B}} u_0 + \overline{\boldsymbol{B}} u_1 \\ x_2 &= \overline{\boldsymbol{A}}^{\boldsymbol{2}}\overline{\boldsymbol{B}} u_0 + \overline{\boldsymbol{A}}\overline{\boldsymbol{B}} u_1 + \overline{\boldsymbol{B}} u_2 \end{aligned} \tag{4}$$

$$\begin{aligned} y_0 &= \overline{\boldsymbol{C}}\overline{\boldsymbol{B}} u_0 \\ y_1 &= \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}\overline{\boldsymbol{B}} u_0 + \overline{\boldsymbol{C}}\overline{\boldsymbol{B}} u_1 \\ y_2 &= \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}^{\boldsymbol{2}}\overline{\boldsymbol{B}} u_0 + \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}\overline{\boldsymbol{B}} u_1 + \overline{\boldsymbol{C}}\overline{\boldsymbol{B}} u_2 \end{aligned} \tag{5}$$

Equations (4) and (5) can be vectorized into the convolutional form given by Equation (6), allowing recursive computations to be expressed in terms of a convolution kernel as shown in Equation (7):

$$\begin{gathered} y_k = \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}^{\boldsymbol{k}}\overline{\boldsymbol{B}} u_0 + \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}^{\boldsymbol{k-1}}\overline{\boldsymbol{B}} u_1 + \cdots + \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}\overline{\boldsymbol{B}} u_{k-1} + \overline{\boldsymbol{C}}\overline{\boldsymbol{B}} u_k \\ y = \overline{\boldsymbol{K}} * u \end{gathered} \tag{6}$$

$$\begin{gathered} \overline{\boldsymbol{K}} \in \mathbb{R}^L \coloneqq \mathcal{K}_L(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \overline{\boldsymbol{C}}) \coloneqq (\overline{\boldsymbol{C}}\overline{\boldsymbol{A}}^i\overline{\boldsymbol{B}})_{i\in[L]} = \\ (\overline{\boldsymbol{C}}\overline{\boldsymbol{B}}, \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}\overline{\boldsymbol{B}}, \dots, \overline{\boldsymbol{C}}\overline{\boldsymbol{A}}^{L-1}\overline{\boldsymbol{B}}) \end{gathered} \tag{7}$$

where the kernel $\overline{\boldsymbol{K}}$ is referred to as the SSM convolution kernel. Given a known kernel $\overline{\boldsymbol{K}}$, the expression in Equation (6) represents a non-circular convolution operation, calculable with high efficiency through FFT methods [11,40]. This efficient parallel computation capability gives SSMs a significant advantage when processing extremely long sequences.

2.2 Deep Structured State Space Models

For further elaborates on the theoretical evolution of state space models. We first introduce the HiPPO theory for modeling long-range dependencies, along with its orthogonal variants. Then, we focus on the structural evolution of SSMs within the deep learning framework, analyzing the design principles, computational characteristics, and practical implementation strategies of representative models such as S4 [11], S4D [26], and DSS.

2.2.1 HiPPO Structure

To equip state space models with the ability to handle infinitely long historical information, Gu et al. proposed a theoretical framework known as High-order Polynomial Projection Operators(HiPPO) [15]. Essentially, HiPPO constructs a set of state variables in the

continuous-time domain that optimally approximate the projection of the historical input function $f(t)$ onto a certain function space. The approach's mathematical formulation can be represented as Equation (8):

$$\min_{c_0,\ldots,c_{N-1}} \int_0^t \omega_t(\tau)|f(\tau) - \sum_{n=0}^{N-1} c_n(t)\phi_n(\tau)\,|^2 d\tau \tag{8}$$

Here, $\omega_t(\tau)$ is a weighting function, and $\{\phi_n(\tau)\}$ is a set of orthogonal basis functions. The corresponding projection coefficients $c_n(t)$ evolve according to a first-order linear differential equation shown in Equation (9):

$$\frac{d}{dt}c(t) = \boldsymbol{A_t}c(t) + b_t f(t) \tag{9}$$

where $\boldsymbol{A_t}$ is the HiPPO state matrix. By choosing different basis functions and weighting schemes, several HiPPO variants have been derived including the following two categories:

1. HiPPO-LegS (Legendre): Uses normalized Legendre polynomials as basis functions with weighting $\omega_t(\tau) = 1$, projecting the function onto the interval [0,t]. The resulting state matrix is an asymmetric dense matrix with excellent spectral coverage properties.

2. HiPPO-LagT (Laguerre): Utilizes Laguerre functions as the basis, employing exponentially decaying weights $\omega_t(\tau) = e^{-\tau}$, making it more suitable for systems sensitive to recent inputs and rapidly forgetting distant inputs.

The two variants emphasize different mechanisms for retaining temporal information: LegS equally preserves long-term history, while LagT emphasizes the importance of recent inputs. In SSMs, the HiPPO matrix constructed from the LegS approach is most commonly employed. LSSL found that merely replacing the random matrix $\boldsymbol{A}$ in SSM with the matrix defined in Equation (10) could improve accuracy rates for sequential MNIST tests increasing from 60% to 98%. The element at row a and column b of matrix $\boldsymbol{A}$ is given by:

$$\boldsymbol{A_{ab}} = \begin{cases} (2a+1)^{\frac{1}{2}}(2b+1)^{\frac{1}{2}} & \text{if a} > \text{b} \\ a+1 & \text{if a} = \text{b} \\ 0 & \text{if a} < \text{b} \end{cases} \tag{10}$$

The matrix features eigenvalues concentrated in the complex plane's left section proximate to the imaginary axis, producing spectral qualities that facilitate both extended memory retention and quick dynamic responses. However, as the HiPPO matrix is typically

dense and non-normal, directly computing the convolution kernel $\boldsymbol{K} = \{\boldsymbol{CA}^k u\}_{k=0}^{L-1}$ via its state transition equations in a neural network incurs a computational complexity of $O(N^2 L)$ and space complexity of $O(NL)$, significantly limiting the model's applicability to long sequences. Consequently, structured design and efficient implementations have become central to subsequent model developments.

2.2.2 Structured State Space Models and Variants

To address computational bottlenecks, Gu et al. introduced a diagonal low-rank (DPLR) structure for parameterizing the state matrix $\boldsymbol{A}$ in the S4 model, formulated as Equation (11):

$$\boldsymbol{A} = \boldsymbol{\Lambda} - \boldsymbol{P}\boldsymbol{P}^* \tag{11}$$

Here, $\boldsymbol{\Lambda} \in \mathbb{C}^{N\times N}$ is a diagonal matrix containing the eigenvalues of the HiPPO matrix, and $\boldsymbol{P} \in \mathbb{C}^{N\times R}$ is a low-rank matrix of rank $\mathcal{R}$. This structure offers three computational advantages:

1. Avoids direct computation of matrix powers by estimating the convolution kernel via generating functions, thus circumventing numerical instabilities associated with exponential calculations;
2. Utilizes a Cauchy kernel to simplify computations by converting $e^{\boldsymbol{A}\Delta}$ into an analytical form within the complex domain;
3. Employs the Woodbury identity to optimize inversion, reducing $\boldsymbol{A}^{-1}$ computations to the inversion of a diagonal matrix and significantly lowering complexity to $O(N + R^2)$

S4 is the first deep state space model to adopt DPLR parameterization [11]. It employs Zero-Order Hold (ZOH) discretization techniques to transform continuous state expressions into discrete sequence models suitable for training, as Equation (12) illustrates:

$$\begin{aligned} \overline{\boldsymbol{A}} &= \boldsymbol{e}^{\boldsymbol{A}\Delta} \\ \overline{\boldsymbol{B}} &= \boldsymbol{A}^{-1}(\boldsymbol{e}^{\boldsymbol{A}\Delta} - \boldsymbol{I})\boldsymbol{B} \end{aligned} \tag{12}$$

Here, Δ denotes the time step size. The core innovation of S4 lies in maintaining the expressive capability of the HiPPO structure while introducing numerical optimization strategies to improve training efficiency. This model significantly outperforms traditional RNN and Transformer architectures across various long-sequence tasks [41].

S4D is a simplified variant based on S4 [26], designed by omitting the low-rank component from the DPLR structure and retaining only the diagonal matrix $\boldsymbol{\Lambda}$, as follows:

$$\boldsymbol{A} = \boldsymbol{\Lambda} \tag{13}$$

This simplification provides the following advantages:

1. Reduces computational complexity from $O(N + R^2)$ to $O(N)$, particularly beneficial for high-dimensional states and long sequences;

2. Discretization can be directly represented as $\overline{\boldsymbol{A}} = diag(e^{\lambda_1 \Delta}, \ldots, e^{\lambda_N \Delta})$, avoiding complex matrix exponential computations;

3. The diagonal structure naturally facilitates parallel training and seamlessly integrates with GPU-accelerated matrix operations.

While forgoing certain low-dimensional memory components, S4D nonetheless delivers equivalent or enhanced results compared to the original S4 across various benchmarks, positioning it as an exemplary model in the category of computationally efficient state space architectures [11,26].

The DSS model is structurally similar to S4D but introduces more flexible initialization mechanisms and regularization techniques, such as optional HiPPO-based or random initialization, alongside regularization techniques like normalized batches and random node deactivation in the training phase, thereby enhancing the stability and generalization ability of the model.

2.3 Attention Mechanism

The fundamental principle behind self-attention is to allow the model to dynamically determine which positions to focus on when processing each position within a sequence [6]. Standard attention approaches work by transforming relationships between queries and assembled key-value elements into resultant vectors, maintaining vector representations throughout the process [45]. Final outcomes represent value aggregations where contributory weights stem from assessing query-key element correspondence [40-42].

For sequence inputs $X \epsilon \mathbb{R}^{L \times d}$ with length L and feature dimensionality d, attention processing begins by generating Query, Key, and Value components via separate linear mappings:

$$Q = XW^Q, K = XW^K, V = XW^V \tag{14}$$

Here, $W^Q, W^K \in \mathbb{R}^{d \times d_k}$ and $W^V \in \mathbb{R}^{d \times d_v}$ are learnable parameter matrices, and d_k, d_v are the dimensions of queries, keys and values, respectively.

The calculation sequence for attention operations can be expressed mathematically as shown in Equation (15):

$$\text{Attention(Q,K,V)} = \text{softmax}(\frac{\boldsymbol{QK^T}}{\sqrt{\boldsymbol{d_k}}})\text{V} \tag{15}$$

In this equation, $QK^T \in \mathbb{R}^{L\times L}$ computes the similarity between every pair of positions in the sequence. The scaling factor $\frac{1}{\sqrt{d_k}}$ stabilizes gradients, and the softmax function normalizes these scores into probability distributions [46]. The system then applies these probability values as weighting factors to combine elements from the value matrix V, producing the resulting representation.

2.4 Gating Mechanism

The gating mechanism is a crucial component in deep sequential models, significantly enhancing their ability to handle long-sequence data through selective control of information flow. The core concept of gating mechanisms involves controlling information flow via learnable gating functions [16]. Mathematically, gating units dynamically regulate state updates and outputs through differentiable gating signals, enabling the model to selectively retain historical information or incorporate new inputs at each time step.

Gating signals are commonly generated using the sigmoid function σ, which compresses arbitrary real values into the interval (0,1), formally represented as Equation (16):

$$\delta(x) = \frac{1}{1+e^{-x}} \tag{16}$$

Sigmoid output values can be conceptualized as signifying permeability levels, with zero indicating complete blockage, one representing total passage, and decimal values showing partial transmission [47]. Through this continuous gating mechanism, gating units achieve smooth information control, allowing networks to selectively update and preserve state information while maintaining gradient flow. The computation of gating signals typically depends on the current input and previous state, as shown in Equation (17):

$$g_t = \delta(W_g x_t + U_g h_{t-1} + b_g) \tag{17}$$

Here, x_t is the current input, h_{t-1} is the previous state, and W_g, U_g, and b_g are learnable parameters.

2.5 Fast Fourier Transform

In long-sequence modeling, frequency domain analysis provides a complementary perspective capable of revealing global periodic structures that are difficult to capture through time-domain analysis alone [48-50]. The Fast Fourier Transform (FFT) represents an optimized computational approach for calculating DET [50]. Computational demands are notably lowered as this method utilizes the natural symmetry and cyclical characteristics within DFT operations.

Specifically, for a sequence of length $N = 2^m$, elements at even and odd indices can be processed separately as illustrated in Equation (18):

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-\frac{j2\pi kn}{N}} = \sum_{r=0}^{N/2-1} x[2r] e^{-\frac{j2\pi k(2r)}{N}} + \sum_{r=0}^{N/2-1} x[2r+1] e^{-\frac{j2\pi k(2r+1)}{N}} \tag{18}$$

This further simplifies to:

$$X[k] = X_{even}\left[k \bmod \frac{N}{2}\right] + e^{-j2\pi k/N} X_{odd}\left[k \bmod \frac{N}{2}\right] \tag{19}$$

Here, X_{even} and X_{odd} denote the DFTs of elements at even and odd indices, respectively. Through this recursive decomposition, FFT reduces the computational complexity to O(NlogN).

Time-domain and frequency-domain representations offer two complementary views for observing the same signal, each excelling at capturing different types of features and structures [49]. Time-domain representations directly reflect signal variations over time, making them suitable for analyzing local features, transient behaviors, and temporally localized events. Conversely, frequency-domain representations reveal periodic structures and frequency compositions, facilitating the identification of long-term patterns, periodicity, and harmonic relationships.

3 Methodology

3.1 Model Architecture

The Channel-aware Structured State Space Sequence model (CS4) utilizes a combined approach that incorporates both time-domain and frequency-spectrum analysis to detect underlying patterns within multi-channel sequential temporal data. Its design concept is built upon three core ideas: (1) utilizing S4 to efficiently handle long-range sequence dependencies; (2) capturing periodicity and frequency characteristics via frequency-domain analysis; and (3) performing channel-aware feature extraction to account for dependencies within multi-channel data.

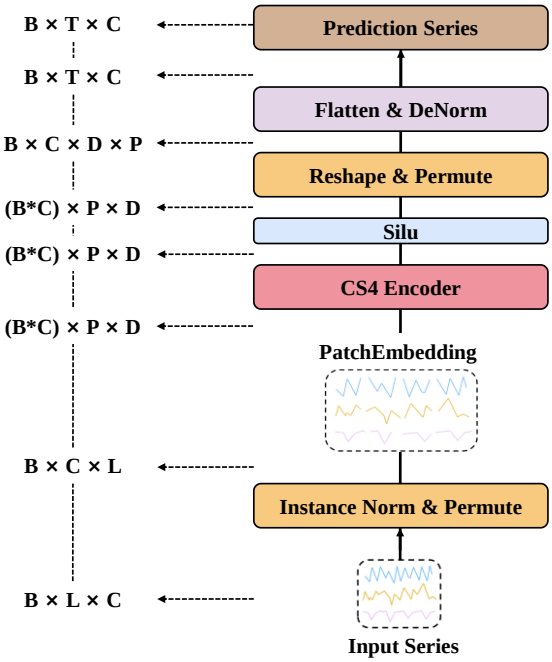


Figure 3. Overall architecture of Channel-aware Structured State Space Sequence model.

As shown in figure 3, our framework employs a comprehensive encoding-prediction architecture. Information processing initiates with raw sequential inputs and advances through conversion phases toward final projections. Input sequences are structured with dimensions [B, L, C], where B signifies batch quantity, L indicates temporal length, and C represents feature channels.

Initially, data undergoes Instance Normalization followed by dimension permutation, resulting in a shape of [B, C, L]. Instance Normalization serves two main purposes: first, it removes scale differences among channels, making the data distribution across channels more

consistent, thus reducing training difficulty; second, this approach strengthens the framework's adaptability to diverse data patterns, especially improving resilience when handling the frequently encountered non-stationarity in temporal sequences.

Following the normalization process, information flows to the PatchEmbedding component, where temporal sequences are segmented into distinct time windows before transformation into rich feature representations. Specifically, the data shaped [B,C,L] is segmented via sliding windows and embedded, converting it to [B×C,P,D], with P indicating patch count and D referring to the dimensionality of features. Inspired by patch segmentation from vision Transformers, this design enables local feature extraction from time series, enabling simultaneous recognition of nearby structures and distant relationships. In contrast to conventional sliding-window techniques, the PatchEmbedding approach combines temporal segmentation with transformation into rich feature dimensions via trainable linear mappings, enhancing representational capabilities.

Subsequently, the data enters the CS4 Encoder, the core component responsible for extracting deep temporal and frequency-domain features. The CS4 Encoder maintains the input shape [B×C,P,D], internally implementing complex dual-path feature extraction described in detail in Section 3.2. After the encoder’s output, a Sigmoid Linear Unit (SiLU) activation function introduces non-linear transformation, further strengthening the model’s representational capacity.

After activation, data undergoes Reshape and Permute operations to rearrange into the shape [B,C,D,P], separating batch and channel dimensions in preparation for subsequent channel-specific feature processing. Then, a Flatten operation transforms the data into [B,T,C], where T is the target prediction length. Finally, data passes through a de-normalization (DeNorm) procedure, mapping features back to the original data distribution space, thus generating the final predicted sequence [B,T,C].

3.2 CS4 Encoder Components

The CS4 Encoder, a core component of the model, comprises multiple cascaded CS4 Blocks responsible for extracting deep temporal and frequency-domain feature representations from the input sequence. As illustrated in figure 4, each CS4 Block employs a dual-path architecture, processing the PatchEmbedding output [B×C,P,D] simultaneously through the time-branch and frequency branch to extract features, subsequently integrating these two sets of features via a gating mechanism. This architectural approach allows the system to thoroughly analyze temporal data characteristics across both time and frequency domains.

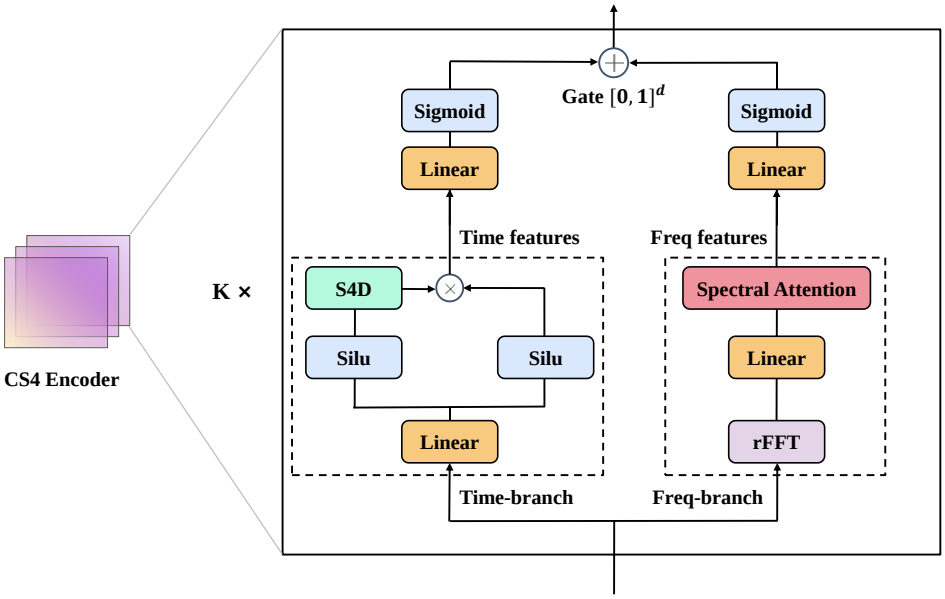


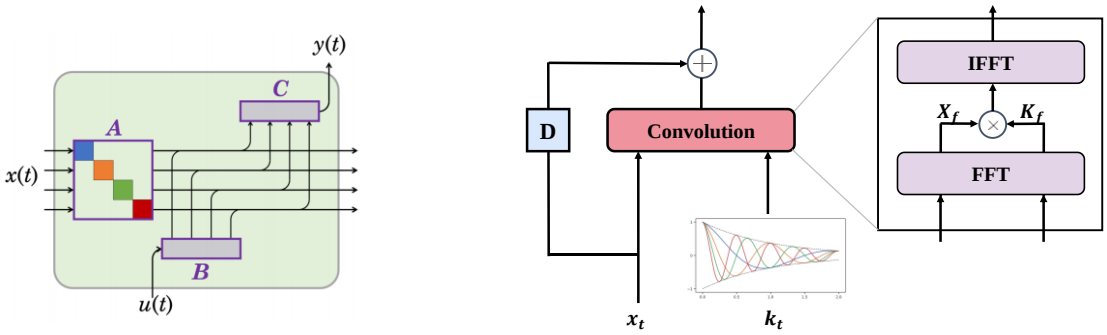
Figure 4. Overall architecture of the CS4 Encoder(time-branch and freq-branch).

3.2.1 Temporal Feature Extraction

In the CS4 model, the time-branch is responsible for capturing temporal dependencies. Its structural design is based on structured state space models and incorporates diagonal parameterization to enhance modeling efficiency and simplicity of implementation. The framework shown in figure 4 reveals how, once local temporal windows are projected into rich feature dimensions, this enhanced data simultaneously traverses two distinct processing routes: time-branch and freq-branch. Within the time-branch, the S4D module is introduced to effectively model long-range temporal dependencies. This branch achieves global modeling capability through a convolution kernel composed of exponential basis functions, complemented by activation and gating mechanisms to output structured temporal feature representations.

Structurally, the time-branch receives tensor representations from the PatchEmbedding layer with a shape of $[B \cdot C, P, D]$, where B is the batch size, C is the number of channels, P is the number of patches, and D is the feature dimension. The input first undergoes a linear projection to increase dimensionality, then splits into two parallel paths: one path includes a SiLU activation followed by the core S4D convolutional layer, and the other path uses only a SiLU activation. The outputs of these two paths are multiplied element-wise and then passed through a linear layer, resulting in the final time-branch feature output. This design introduces a multiplicative gating mechanism, enabling the model to adaptively adjust the importance of

each feature component while preserving the nonlinear expressiveness provided by the activation function.



S4D Recurrent view

S4D Convolution view

Figure 5. Computational framework of the S4D module and its FFT-based acceleration.

Figure 5 provides a visual representation of the time-branch's organizational structure and essential computational processes in the CS4 framework. On the left is a decoupled subsystem of the structured state space system, where the state transitions for each channel are governed by the diagonal state matrix $A = diag(A_0, A_1, \dots, A_{N-1})$. The corresponding convolution kernel in discrete time takes the Vandermonde form shown in Equation (20):

$$\boldsymbol{K} = [\boldsymbol{B_0C_0}\ \ \boldsymbol{B_1C_1}\ \ \dots\ \ \boldsymbol{B_{N-1}C_{N-1}}] \begin{bmatrix} 1 & \boldsymbol{A_0} & \boldsymbol{A_0^2} & \cdots & \boldsymbol{A_0^{L-1}} \\ 1 & \boldsymbol{A_1} & \boldsymbol{A_1^2} & \cdots & \boldsymbol{A_1^{L-1}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \boldsymbol{A_{N-1}} & \boldsymbol{A_{N-1}^2} & \cdots & \boldsymbol{A_{N-1}^{L-1}} \end{bmatrix} \tag{20}$$

As demonstrated in the figure's rightmost section, kernel calculations benefit from computational efficiency through FFT-IFFT implementation: the input signal x_t and kernel k_t are separately transformed into the frequency domain using FFT, then multiplied point-wise (Hadamard Product), and finally transformed back into the time domain using IFFT to complete the convolution operation.

Within the time-branch, we select the Sigmoid Linear Unit(SiLU) as activation function, and defined it as $f(x) = x \cdot \sigma(x)$, where σ denotes the sigmoid function. The rationale for choosing SiLU over ReLU or tanh is as follows:

1. SiLU is a smooth nonlinear function, beneficial for gradient propagation;
2. its non-monotonic nature enhances model expressiveness;

3. it allows a small number of negative values, facilitating the transmission of negatively correlated features in temporal data;

4. it is inherently compatible with the multiplicative gating mechanism employed in the time-branch.

3.2.2 Spectral Feature Extraction

Unlike the time-branch, which focuses on capturing temporal dependencies, the freq-branch directly extracts periodic characteristics and frequency structures from sequences via spectral analysis, providing complementary informational representations to the model. As shown in figure 4, the processing pipeline of the freq-branch mainly includes real Fast Fourier Transform (rFFT), linear mapping, and a Spectral Attention mechanism.

The rFFT leverages the conjugate symmetry of real-valued sequence spectra shown in Equation (21), requiring only the first $\frac{N}{2} + 1$frequency components to obtain complete spectral information.

$$X[N-k] = X[k]^*, \quad k = 1,2,\dots,\frac{N}{2} \tag{21}$$

This effectively reduces computational cost and memory overhead, thereby improving the overall inference efficiency of the model. More importantly, the frequency-domain representation directly reveals the energy distribution of various frequency components in time series data, providing more interpretable input features for subsequent attention mechanisms in frequency-domain modeling.

A distinctive contribution in the frequency analysis segment is the Spectral Attention technique, intentionally developed to interpret relationships among features after spectral transformation, with methodological details presented in Section 2.3. The formulation appears similar to that of standard attention mechanisms; however, Spectral Attention differs from standard attention in the following ways:

1. Operational domain: The Spectral Attention mechanism functions within frequency space instead of temporal dimensions, emphasizing relationships between spectral elements rather than sequential positions;

2. Relational pattern: While standard attention mechanisms primarily capture dependencies among different positions in a sequence, Spectral Attention emphasizes interaction patterns within the spectrum, such as harmonic structures, frequency-band energy distributions, and frequency resonance.

3.2.3 Feature Fusion

After extracting features from different perspectives, the time-branch and freq-branch are integrated through a gating fusion mechanism to form the final feature representation. This fusion process can be formalized as Equation (22):

$$\begin{aligned} g_{time} &= \sigma(Linear(h_{time})) \\ g_{freq} &= \sigma(Linear(h_{freq})) \\ g &= g_{time} + g_{freq} \\ h_{out} &= g \odot h_{time} + (1 - g) \odot h_{freq} \end{aligned} \tag{22}$$

Here, h_{time} and h_{freq} denote the output features from the time-branch and freq-branch, respectively; σ represents the sigmoid activation function; $g \in [0,1]^d$ is the fusion gating factor; $\odot$ denotes the Hadamard product; and h_{out} is the final fused output.

3.3 Model Effectiveness Analysis

This section evaluates the effectiveness of the CS4 model from three perspectives: approximation accuracy, computational complexity, and parameter efficiency.

3.3.1 Accuracy Analysis

To rigorously validate the theoretical foundation of the proposed CS4 architecture, we conduct a comprehensive analysis of its universal approximation capability and frequency response characteristics.

Let $X \subset \mathbb{R}^{L \times d_{in}}$ and $Y \subset \mathbb{R}^{H \times d_{out}}$ denote compact sets endowed with the uniform norm $\| z \|_\infty = max_i |z_i|$. Denote $\mathcal{C}(X, Y)$ as the space of continuous mappings from *X* to *Y*.

Theorem For any $f \in \mathcal{C}(X, Y)$ and $\varepsilon > 0$, there exists a parameter set Θ of the CS4 model such that:

$$\sup_{x \epsilon X} \| f(x) - CS4_\Theta(x) \|_\infty < \varepsilon \tag{23}$$

Proof Firstly, the CS4 output can be decomposed into three components: $CS4_\Theta(x) = G_{\theta_g}(T_{\theta_t}(x), F_{\theta_f}(x))$, where $T_{\theta_t}, F_{\theta_f}$ and G_{θ_g} represent time-branch, frequency-branch and fusion gating unit. For time-branch approximation, by the universal approximation theorem for neural networks, if the activation function is non-polynomial and bounded, the temporal branch T_{θ_t} can approximate any continuous function $f_t(x)$ on compact sets. Specifically, for $\varepsilon/3>0$, there exists θ_t such that:

$$\sup_{x\epsilon X}\|f_t(x) - T_{\theta_t}(x)\|_\infty < \varepsilon/3 \tag{24}$$

For frequency-branch approximation, the discrete Fourier transform is a unitary operator. Let $\hat{f} = F \circ f$. By the universal approximation theorem, there exists θ_f such that:

$$\sup_{x\epsilon X}\|\hat{f}(x) - F_{\theta_f}(x)\|_\infty < \varepsilon/3 \tag{25}$$

By Parseval's theorem and the continuity of the inverse DFT, the time-domain error satisfies:

$$\|f(x) - F^{-1} \circ F_{\theta_f}(x)\|_\infty \leq C \cdot \varepsilon/3 \tag{26}$$

where C is the operator norm of the inverse DFT. Adjusting ε absorbs C into $\varepsilon/3$.

The fusion gating G_{θ_g}, implemented as a two-layer MLP with Sigmoid activation, can approximate the ideal fusion function $g^*(u, v) = f(x)$ due to Cybenko's theorem. Thus:

$$\sup_{(u,v)}\|g^*(u, v) - G_{\theta_g}(u, v)\|_\infty < \varepsilon/3 \tag{27}$$

By the triangle inequality:

$$\|f(x) - CS4_\Theta(x)\|_\infty \leq \sum_{\varepsilon=j}^{3} \varepsilon\,/3 = \varepsilon \tag{28}$$

This completes the proof.

3.3.2 Computational Complexity Analysis

The time complexity of the CS4 model consists of three components: the time branch, the frequency branch, and the fusion mechanism. The core computation of the time branch lies in the structured convolution operations of the S4D layer. For an input sequence of length L, the computational process can be decomposed as follows:

1. State space discretization: Initialization of the diagonal state matrix $\boldsymbol{A} \in \mathbb{R}^N$, with a complexity of $\mathcal{O}(N)$;
2. FFT-accelerated convolution: The convolution kernel $\boldsymbol{K} \in \mathbb{R}^L$ is processed in the spectral domain using FFT algorithms, with a complexity of $\mathcal{O}(L\,logL)$;
3. Linear projection: Dimensional mapping between input or output features and the state space, with a complexity of $\mathcal{O}(LN)$.

Consequently, the temporal branch's aggregate computational complexity equals:

$$\mathcal{O}(N) + \mathcal{O}(L\,logL) + \mathcal{O}(LN) = \mathcal{O}(L\,logL + LN) \tag{29}$$

The complexity of the frequency branch mainly arises from the real-valued FFT transformation $\mathcal{O}(L\,logL)$ and the spectral attention mechanism. As spectral attention leverages frequency-domain sparsity, its computational cost is $\mathcal{O}(L\,logL \cdot d)$, where d is the feature dimension. The gated weighting operation in the fusion mechanism, a two-layer MLP, requires only linear complexity $\mathcal{O}(Ld)$. Consequently, the upper bound of the overall time complexity of CS4 is:

$$\mathcal{O}(L\,logL + LN) + \mathcal{O}(L\,logL \cdot d) + \mathcal{O}(Ld) = \mathcal{O}(L\,logL \cdot d + LN) \tag{30}$$

When d and N $\ll$ L, the feature and state dimensions are much smaller than the sequence length, the dominant term simplifies to $\mathcal{O}(L\,logL)$. In contrast to the attention system employed in conventional Transformer architectures $\mathcal{O}(L^2 d)$, the asymptotic advantage of CS4 can be quantified via limit analysis:

$$\lim_{L\to\infty} \frac{L\,logL \cdot d}{L^2 d} = \lim_{L\to\infty} \frac{logL}{L} = 0 \tag{31}$$

This result indicates that the computational efficiency of CS4 improves superlinearly with increasing sequence length.

3.3.3 Parameter Complexity Analysis

The parameter complexity of CS4 is completely decoupled from the sequence length and is solely determined by the model's structural hyperparameters.

For parameters of the S4D layer, the diagonal state matrix $\boldsymbol{A} \in \mathbb{R}^N$, input projection $\boldsymbol{B} \in \mathbb{R}^N$, and output projection $\boldsymbol{C} \in \mathbb{R}^N$, each with complexity $\mathcal{O}(N)$. Linear transformation weights from input features to the state space $W_{in} \in \mathbb{R}^{d\times N}$, with a complexity of $\mathcal{O}(Nd)$. The total parameter count is:

$$\mathcal{O}(N) + \mathcal{O}(Nd) = \mathcal{O}(Nd) \tag{32}$$

For parameters of the frequency branch, query, key, and value projection matrices of the spectral attention mechanism $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$, each with a complexity of $\mathcal{O}(d^2)$. Output projection layer $W_{out} \in \mathbb{R}^{d\times d}$, with a complexity of $\mathcal{O}(d^2)$. Summing all components, the total parameter complexity of CS4 is:

$$\mathcal{O}(Nd) + \mathcal{O}(d^2) + \mathcal{O}(d^2) = \mathcal{O}(Nd + d^2) \tag{33}$$

Compared to the standard Transformer, which has a parameter count of $\mathcal{O}(d^2)$, CS4 maintains similar parameter efficiency when N∼d. The efficiency ratio is:

$$\eta = \frac{\mathcal{O}(Ld^2)}{\mathcal{O}(L\,logL \cdot d)} = \mathcal{O}\left(\frac{L}{\log L}\right) \tag{34}$$

3.4 Model Generation Process

To enhance transparency regarding the fundamental processes within the proposed CS4 framework and to support replication efforts, this section presents three pieces of high-level pseudocode: (1) the overall training loop, (2) the forward-inference routine, and (3) the internal computation of a single CS4Encoder block. Variables and function names match those used in Section 3.1–3.2.

Algorithm 1 outlines the training procedure for the CS4 model within an encoder–decoder framework. The system processes training, validation, and testing datasets as inputs and works to discover optimal configuration parameters through the Adam optimization algorithm with MSE as the evaluation criterion.During each epoch, the model performs a forward pass to produce predictions, computes the MSE loss, back-propagates gradients and updates parameters accordingly. A learning-rate scheduler adjusts the optimizer's step size over time, and an early-stopping mechanism—triggered by validation-set performance—prevents overfitting. Throughout training, the algorithm continually tracks validation loss, saves the parameters corresponding to its minimum value, and ultimately returns this best parameter set to ensure maximal generalization capability.

Furthermore, algorithm 2 presents the forward-inference pipeline of the CS4 model. Given a multi-channel input time series, the procedure first applies instance normalization to each channel. The normalized data are then reshaped and embedded into patches via a dimension-reordering and patch-embedding step. These embeddings are fed into the CS4 encoder, whose dual-branch design concurrently captures both temporal and spectral features. The encoder's output is subsequently passed through a flattened head network to generate raw predictions, which are finally restored to the original data scale by inverse normalization, yielding the model's predicted sequence.

The CS4 block in algorithm 3 employs a dual branch architecture. The temporal branch leverages the S4D module to capture long range dependencies across time, while the spectral branch extracts periodic features by applying a Fourier transform followed by a spectral

attention mechanism. The two branches' representations are then merged via a gating mechanism, wherein the fusion weights are dynamically conditioned on the input feature values.

Algorithm 1: CS4 Training Procedure

INPUT:

$D_{train} = \{(X_{enc}, T_{enc}, X_{dec}, T_{dec})\}$, D_{val}, D_{test} ⟶ training dataset

$\theta \leftarrow \theta_0$ ⟶ initial model parameters

η, E, B

OUTPUT:

θ^* ⟶ best-found parameters

Initialize CS4(θ), Optimizer ← Adam(θ, lr= η), Scheduler
best_val_loss ← +∞
for epoch = 1 to E **do**
  train_loss ← 0
  for each batch b ∈ D_{train} of size B **do**
    (X_{enc}, T_{enc}, X_{dec}, T_{dec}) ← b
    X_{dec}_input ← ShiftRight(X_{dec})
    Ŷ ← CS4_Forward(X_{enc}, T_{enc}, X_{dec}_input, T_{dec})
    ℓ ← MSE(Ŷ, X_{enc}) ⟶ mean squared error loss
    Optimizer.zero_grad()
    ℓ.backward() ⟶ backpropagation
    Optimizer.step() ⟶ update θ
    train_loss ← train_loss + ℓ
  end for
  Scheduler.step()
  val_loss ← Evaluate(CS4, D_{val}) ⟶ loss on validation set
  if val_loss < best_val_loss **then**
    best_val_loss ← val_loss
    SaveCheckpoint(θ) ⟶ store current best
  end if
  if EarlyStopping(val_loss) **then** break ⟶ stop if no improvement
end for
LoadCheckpoint(θ^*) ⟶ rest
return θ^*

Algorithm 2: CS4 Forward-Inference Routine

INPUT:

$X \in \mathbb{R}^{B\times L\times C}$ ⟶ raw multi-channel time series

T_{enc}, T_{dec} ⟶ positional/time-feature inputs

OUTPUT:

$\hat{Y} \in \mathbb{R}^{B\times T\times C}$ ⟶ predicted sequence

1: $X_{norm} \leftarrow$ InstanceNorm(X) ⟶ channel-wise normalization

2: $X_{perm} \leftarrow$ Permute(X_{norm}, [0,2,1]) ⟶ shape → [B, C, L]

3: Patches ← PatchEmbedding(X_{perm}, T_{enc}) ⟶ [B·C, P, D]

4: Encodings ← CS4Encoder(Patches) ⟶ dual-branch encoder

5: $Y_{flat} \leftarrow$ FlattenHead(Encodings, statistics) ⟶ linear head → [B, C, T]

6: $\hat{Y} \leftarrow$ DeNorm(Y_{flat}t, statistics)

7: **return** $\hat{Y}$

Algorithm 3: CS4Encoder Single Block Computation

INPUT:

$F \in \mathbb{R}^{(B\cdot C)\times P\times D}$ ⟶ patch embeddings

Params = {S4D weights, SpectralAttention weights, fusion weights}

OUTPUT:

$F_{out} \in \mathbb{R}^{(B\cdot C)\times P\times D}$ ⟶ fused feature output

1: $F_1 \leftarrow Linear_1(F)$ ⟶ time-branch

2: $F_2 \leftarrow SiLU(F_1)$

3: $F_{s4} \leftarrow$ S4D(F_2)

4: $F_t \leftarrow Linear_2(F_{s4})$

5: $F_{freq} \leftarrow rFFT(F)$ ⟶ frequency-branch

6: $F_{proj} \leftarrow Linear_3(F_{freq})$

7: $F_{sa} \leftarrow$ SpectralAttention(F_{proj})

8: $F_{IFFT} \leftarrow$ IFFT(F_{sa}) ⟶ back to time domain

9: G ← sigmoid($Linear_4$ (concat(F_t, F_{IFFT}))) ⟶ feature fusion

10: $F_{out} \leftarrow G \odot F_t + (1 - G) \odot F_{IFFT}$ ⟶ gated combination

11: **return** F_{out}

4 Experiements

4.1 Experimental Setup

This study examines model performance across seven open-access temporal datasets, encompassing the ETT, Electricity, Weather and Traffic, which cover multivariate time series forecasting tasks across different domains and scales. Table 1 presents comprehensive statistical information for each dataset:

1. **Electricity Transformer Temperature (ETT)**: Consists of four subsets, ETTh1, ETTh2, ETTm1, and ETTm2, where "h" denotes hourly sampling and "m" denotes minute-level sampling. These datasets contain 7 feature dimensions of electrical transformers, including 6 power load features and oil temperature (OT) as the target value. The ETTh series uses 1-hour granularity sampling with approximately 17,420 time steps, while the ETTm series uses 15-minute granularity sampling with approximately 69,680 time steps.

2. **Electricity:** Records hourly electricity consumption data (in kWh) for 321 customers, totaling 26,304 time steps. The first 320 variables serve as input features to predict the electricity consumption of the 321st customer.

3. **Weather:** Includes a set of 21 weather parameters monitoring conditions such as thermal readings and atmospheric water content, collected in 2020 with a 10-minute sampling interval, totaling 52,696 time steps.

4. **Traffic:** Contains highway occupancy measurements from 862 monitoring sensors, collected hourly between 2015-2016, comprising 17,544 sequential observations.

Table 1. Information about seven datasets used for the experiment.

Dataset	Variates	Timesteps	Granularity	Train:Val:Test
ETTh1	7	17420	1hour	6:2:2
ETTh2	7	17420	1hour	6:2:2
ETTm1	7	69680	15min	6:2:2
ETTm2	7	69680	15min	6:2:2
Electricity	321	26304	1hour	7:1:2
Weather	21	52696	10min	7:1:2
Traffic	862	17544	1hour	7:1:2

To ensure experimental fairness and model training stability, all datasets use fixed-length sliding windows to construct input-output sample pairs. The input window length (look-back window) is fixed at 96 time steps, while the forecast horizons are set to 96, 192, 336, and 720 time steps to evaluate model performance across different prediction lengths. We fix the patch length at 16 and the patch stride at 8. Unless otherwise specified, all error and runtime data in the experiments are obtained by averaging results from 20 experimental runs.

For the 7 complex long-time series forecasting tasks, Mean-Square Error (MSE) is used as the loss function during training, with both MSE and MAE functioning as performance assessment metrics during post-training evaluation and for analyzing representational capabilities of SSMs layers, where for n samples, predicted values $\hat{y}$ and their target values y:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$
$$MAE = \frac{1}{n}\sum_{i=1}^{n} \| y_i - \hat{y}_i \| \tag{35}$$

The sequence models including the original SSMs layers used in the experiments were implemented using Python 3.12 and PyTorch 1.13.1. The experimental data were trained, tested, and collected using a system equipped with a 12th Gen Intel(R) Core(TM) i9-12900H CPU (2.50 GHz), 256 GB of memory, and an NVIDIA A100 GPU provided by Google Colab.

4.2 Comparative Model Setup

The CS4 framework underwent extensive comparison with 5 leading temporal forecasting methodologies, with detailed descriptions presented in Appendix A.

1. **iTransformer** (Liu et al., 2023): An approach reconfigures conventional Transformer architecture by reversing attention functions—employing self-attention for cross-channel relationship modeling while utilizing linear transformations for temporal pattern recognition, offering fresh insights for multi-variable sequential prediction.

2. **PatchTST** (Nie et al., 2022): A patch-level time series forecasting model based on the Transformer architecture that segments time series into multiple patches and processes multivariate time series through a Channel-Independent strategy.

3. **TimesNet** (Wu et al., 2022): A model for temporal data prediction based on convolutional architecture that transforms single-dimensional sequential data into two-

dimensional formats, enabling the detection of both internal pattern relationships and cross-pattern connections through multi-cyclic analysis.

4. **RLinear** (Li et al., 2023): A methodology merging linear functions with RevIN (Reversible Instance Normalization) that implements isolated-channel techniques for multi-feature time sequence projection.

5. **LSTM:** A traditional LSTM neural architecture that serves as a baseline comparison model representing recurrent network approaches.

"For experimental validity, we maintained consistent evaluation protocols across all frameworks, utilizing original default parameters from respective publications while applying uniform data divisions and preparation methods. All systems processed input sequences of 96 steps and generated forecasts spanning four horizons: 96, 192, 336, and 720 steps, matching the CS4 implementation.

4.3 Comparative Model Setup

4.3.1 Parameter Sensitivity Analysis

Model depth is one of the key parameters determining the complexity and expressive capability of neural networks. "To conduct a methodical examination of how varying the number of layers affects the CS4 model's forecasting capabilities, we conducted comparative experiments across different datasets, fixing other parameters while adjusting the number of layers ={2,3,4} and model width d_model={128,256}. According to the detailed results available in Appendix B, figure 6 presents heatmaps of the MSE growth rate for the model across three representative datasets—ETTh1, ETTm1, and Traffic—under different layer configurations.

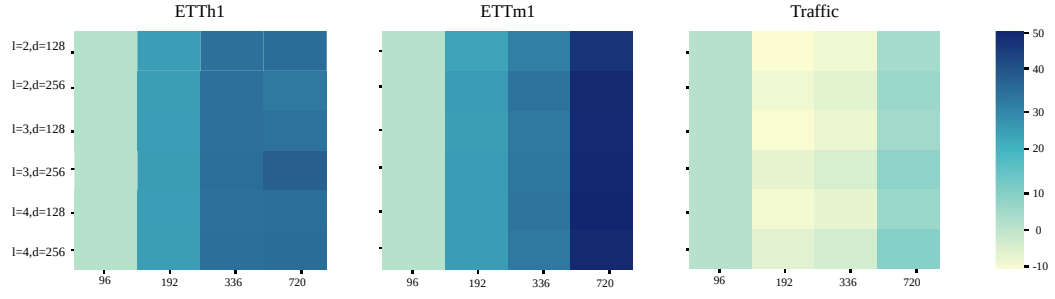


Figure 6. Heat map of MSE of CS4 model with different layer configurations on ETTh1, ETTm1 and Traffic datasets.

The empirical findings suggest a strong correlation between dataset properties and optimal architectural depth. Specifically for ETTh1, modifications to layer quantity demonstrate

negligible impact on short-range predictions, yet when considering long-range forecasting (pred_len=720), the error growth rate of the layer=3 configuration (31.3%) is significantly higher than that of the layer=2 configuration (24.6%), suggesting that excessively deep models may lead to overfitting.

ETTm1 exhibits strong forecast horizon sensitivity, as all tested configurations show error increases surpassing 43% for extended projections, while architectural depth variations produce generally uniform effects. This may stem from its high-frequency sampling characteristics, making the difficulty of long-term prediction significantly greater than the impact brought by parameter adjustments.

The Traffic dataset demonstrates a distinctly different pattern, with errors actually decreasing during medium-term prediction (pred_len=192,336), and this improvement becomes more significant with the increasement of layers and layer=4 configuration performing optimally (-3.1% and -2.0%). This indicates that for data with complex spatiotemporal characteristics, deeper models help capture the intricate patterns therein.

We found that the optimal choice of model depth is highly dependent on data characteristics and prediction length. When d_model=256, the impact of layer number variations becomes more pronounced, indicating that greater model width provides more sufficient expressive space for networks of different depths.

Table 2. Impact of increasing CS4 model layers from 2 to 4 on prediction performance (percentage change in MSE relative to the layer=2 baseline).

Pred_lens & d_model	**ETTh1**	**ETTm1**	**ETTh2**	**ETTm2**	**Electricity**	**Weather**	**Traffic**
96&128	+0.17↑	**-1.66**↓	-0.30↓	-0.37↓	-0.16↓	**-2.65**↓	-1.08↓
96&256	-0.12↓	+0.04↑	-**2.61**↓	**-0.42**↓	**-0.31**↓	-1.42↓	**-2.27**↓
192&128	+0.04↑	-0.06↓	-1.12↓	-0.21↓	-0.15↓	-1.71↓	-0.45↓
192&256	-0.14↓	-0.25↓	**+2.38**↑	+0.23↑	-0.11↓	-0.91↓	-1.15↓
336&128	+0.31↑	+0.31↑	-0.63↓	+0.23↑	-0.12↓	-1.36↓	-0.35↓
336&256	+0.28↑	-1.24↓	-2.31↓	+0.39↑	-0.09↓	-0.67↓	-1.01↓
720&128	**-0.33**↓	**+0.85**↑	+0.73↑	**+0.65**↑	-0.02↓	-0.86↓	-0.36↓
720&256	**+2.56**↑	+0.01↑	-0.63↓	+0.22↑	-0.10↓	-0.75↓	-0.84↓

The best is highlighted in **red** and the runner-up in **blue.**

Model width, referring to the dimension size of hidden layers, is another key factor affecting the expressive capability and parameter count of neural networks. We evaluated how architectural width influences predictive accuracy by examining two different model configurations: d_model={128,256}. Table 2 presents the percentage change in MSE when increasing d_model from 128 to 256 under different layer numbers and prediction lengths. All the data are avaliable in Appendix B.

Experimental results demonstrate that increasing model width enhances prediction performance in most cases, though the magnitude of impact varies across datasets and parameter configurations. On the Traffic and Weatherdataset, the performance improvement from increased model width is most significant, with all configurations showing notable MSE reduction, especially in short-term prediction (pred_len=96) with deeper layers (layer=4), where the MSE decreased by up to 2%. This indicates that for traffic flow data with complex spatiotemporal characteristics, larger model capacity helps capture the inherent structure.

4.3.2 Comparative Analysis with Baseline Models

This section compares the CS4 model with five mainstream baseline models: iTransformer, PatchTST, TimesNet, RLinear, and LSTM. All experiments utilized fixed-length historical data segments (L=96) with evaluation conducted at four distinct forecast ranges: T∈{96, 192, 336, 720}. All experiments were conducted using optimal parameter configurations (layers=4, d_model=256), with Mean Square Error (MSE) and Mean Absolute Error (MAE) as evaluation metrics. Table 3 summarizes each methodology's mean performance figures when tested against five benchmark datasets: ETTh1, ETTh2, ETTm1, ETTm2, and ECL. Top-performing configurations appear in red while second-ranking results are shown in blue. Appendix C contains the comprehensive result set.

Experimental results demonstrate that CS4 exhibits outstanding performance on the MAE metric, achieving the best results on four out of five datasets (ETTm1, ETTm2, ETTh1, ETTh2). For the MSE metric, CS4 performs optimally on the ETTm2 dataset while maintaining competitiveness across other datasets. Notably, on the Electricity dataset containing numerous parallel time series, iTransformer demonstrates superior performance, which may be attributed to its self-attention mechanism's advantage in capturing cross-channel dependencies.

Figure 7 illustrates the MAE performance trends of each model across different prediction lengths. As expected in time series forecasting theory, prediction errors for all models show an upward trend with increasing prediction length. Across ETTm1, ETTh1, and ETTh2 collections, the CS4 approach (depicted by red traces) shows superior performance throughout

all forecast horizons, highlighting its effectiveness for both immediate and distant projections. Regarding ETTm2, despite generally similar performance among tested frameworks, CS4 maintains its edge specifically during extended forecasting (pred_len=720). When examining Electricity data, while iTransformer leads during near-term predictions, CS4 exhibits notable consistency as projection timeframes extend.

Table 3. Performance comparison of CS4 and baseline models across five datasets using MSE and MAE metrics.

Models		**CS4**		**iTransformer**		**PatchTST**		**TimesNet**		**RLinear**		**LSTM**	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ETTm1	96	0.324	0.346	0.334	0.368	0.329	0.367	0.338	0.375	0.355	0.376	0.628	0.556
	192	0.373	0.370	0.377	0.391	0.367	0.385	0.374	0.387	0.391	0.392	0.710	0.606
	336	0.405	0.392	0.426	0.420	0.399	0.410	0.410	0.411	0.424	0.415	0.810	0.650
	720	0.472	0.430	0.491	0.459	0.454	0.439	0.478	0.450	0.487	0.450	0.958	0.726
	Avg	**0.394**	**0.384**	**0.407**	**0.410**	**0.387**	**0.400**	**0.400**	**0.406**	**0.414**	**0.408**	**0.776**	**0.635**
ETTm2	96	0.175	0.254	0.180	0.264	0.176	0.259	0.187	0.267	0.182	0.265	0.219	0.333
	192	0.241	0.298	0.250	0.309	0.242	0.302	0.249	0.309	0.246	0.304	0.374	0.374
	336	0.302	0.336	0.311	0.348	0.305	0.343	0.321	0.351	0.307	0.342	0.463	0.477
	720	0.400	0.392	0.412	0.407	0.402	0.400	0.408	0.403	0.407	0.398	0.830	0.616
	Avg	**0.279**	**0.320**	**0.288**	**0.332**	**0.281**	**0.326**	**0.291**	**0.333**	**0.286**	**0.327**	**0.472**	**0.450**
ETTh1	96	0.385	0.387	0.386	0.405	0.414	0.419	0.384	0.402	0.386	0.395	0.802	0.646
	192	0.443	0.418	0.441	0.436	0.460	0.445	0.436	0.429	0.439	0.424	0.977	0.716
	336	0.491	0.443	0.487	0.458	0.501	0.466	0.491	0.469	0.479	0.446	0.851	0.674
	720	0.494	0.466	0.503	0.491	0.500	0.488	0.521	0.500	0.481	0.470	1.136	0.794
	Avg	**0.453**	**0.429**	**0.454**	**0.448**	**0.469**	**0.455**	**0.458**	**0.450**	**0.446**	**0.434**	**0.941**	**0.708**
ETTh2	96	0.281	0.329	0.297	0.349	0.302	0.348	0.340	0.374	0.288	0.338	0.796	0.603
	192	0.368	0.385	0.380	0.400	0.388	0.400	0.402	0.414	0.374	0.390	0.606	0.536
	336	0.403	0.415	0.428	0.432	0.426	0.433	0.452	0.452	0.415	0.426	0.751	0.598
	720	0.410	0.431	0.427	0.445	0.431	0.446	0.462	0.468	0.420	0.440	0.987	0.665
	Avg	**0.366**	**0.390**	**0.383**	**0.407**	**0.387**	**0.407**	**0.414**	**0.427**	**0.374**	**0.399**	**0.785**	**0.601**
ECL	96	0.192	0.265	0.148	0.240	0.195	0.285	0.168	0.272	0.201	0.281	0.357	0.423
	192	0.194	0.269	0.162	0.253	0.199	0.289	0.184	0.289	0.201	0.283	0.371	0.435
	336	0.208	0.285	0.178	0.269	0.215	0.305	0.198	0.300	0.215	0.298	0.383	0.433
	720	0.250	0.318	0.225	0.319	0.256	0.337	0.220	0.320	0.257	0.331	0.391	0.439
	Avg	**0.211**	**0.284**	**0.178**	**0.270**	**0.216**	**0.304**	**0.193**	**0.295**	**0.219**	**0.298**	**0.375**	**0.433**
1st		**11**	**21**	**4**	**4**	**4**	**0**	**3**	**0**	**3**	**0**	**0**	**0**

The best is highlighted in **red** and the runner-up in **blue.**

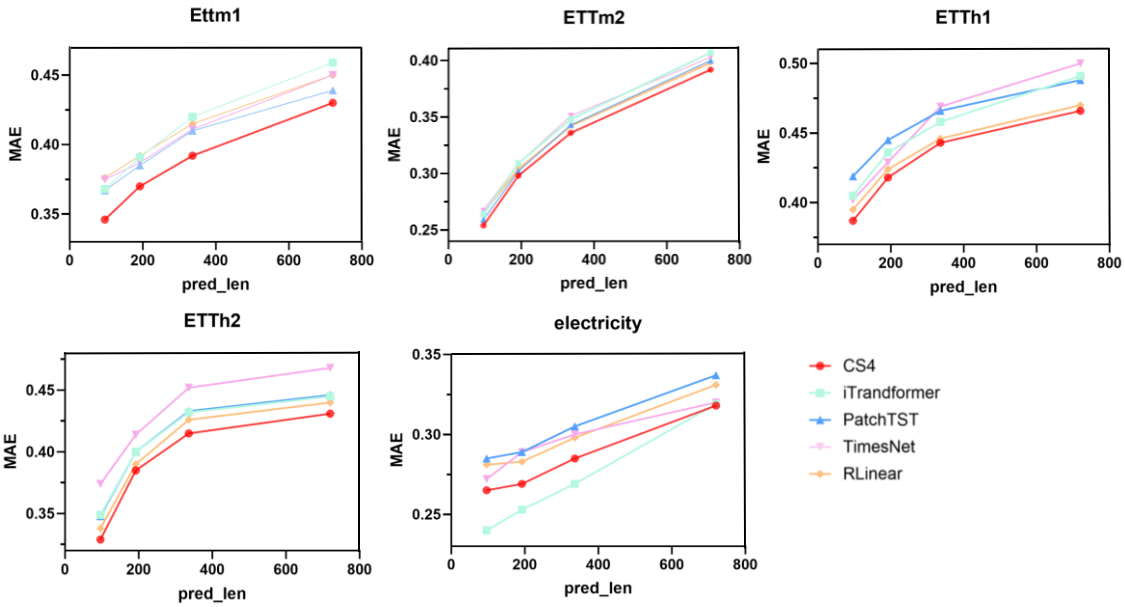


Figure 7. MAE performance trends of CS4 and baseline models across different prediction lengths.

To illustrate CS4's forecasting advantages visually, figures 8 and 9 present comparative prediction outcomes for ETTh2 and ETTm2 datasets. Figure 8 positions first-ranked CS4 results (left) alongside second-ranked RLinear outputs (right) for ETTh2, displaying increasing prediction windows from 96 to 720. Examination reveals CS4's superior pattern recognition capabilities as forecast horizons extend, particularly evident at pred_len=336 and pred_len=720, where CS4 reproduces extrema with higher fidelity while RLinear shows temporal misalignment and reduced oscillation magnitude during distant projections.

A comparative visualization in Figure 9 presents CS4 performance results (left) against PatchTST results (right) for the ETTm2 dataset. The two models perform similarly in short-term prediction (pred_len=96), but as the prediction length increases to 336 and 720, the CS4 model depicts local details more precisely while maintaining the overall morphology of the time series. Particularly at pred_len=336, the PatchTST model insufficiently predicts the low-frequency fluctuations in the 200-300 time point interval, whereas CS4 better captures this characteristic.The forecasting results of other five datasets are visualized in Appendix D.

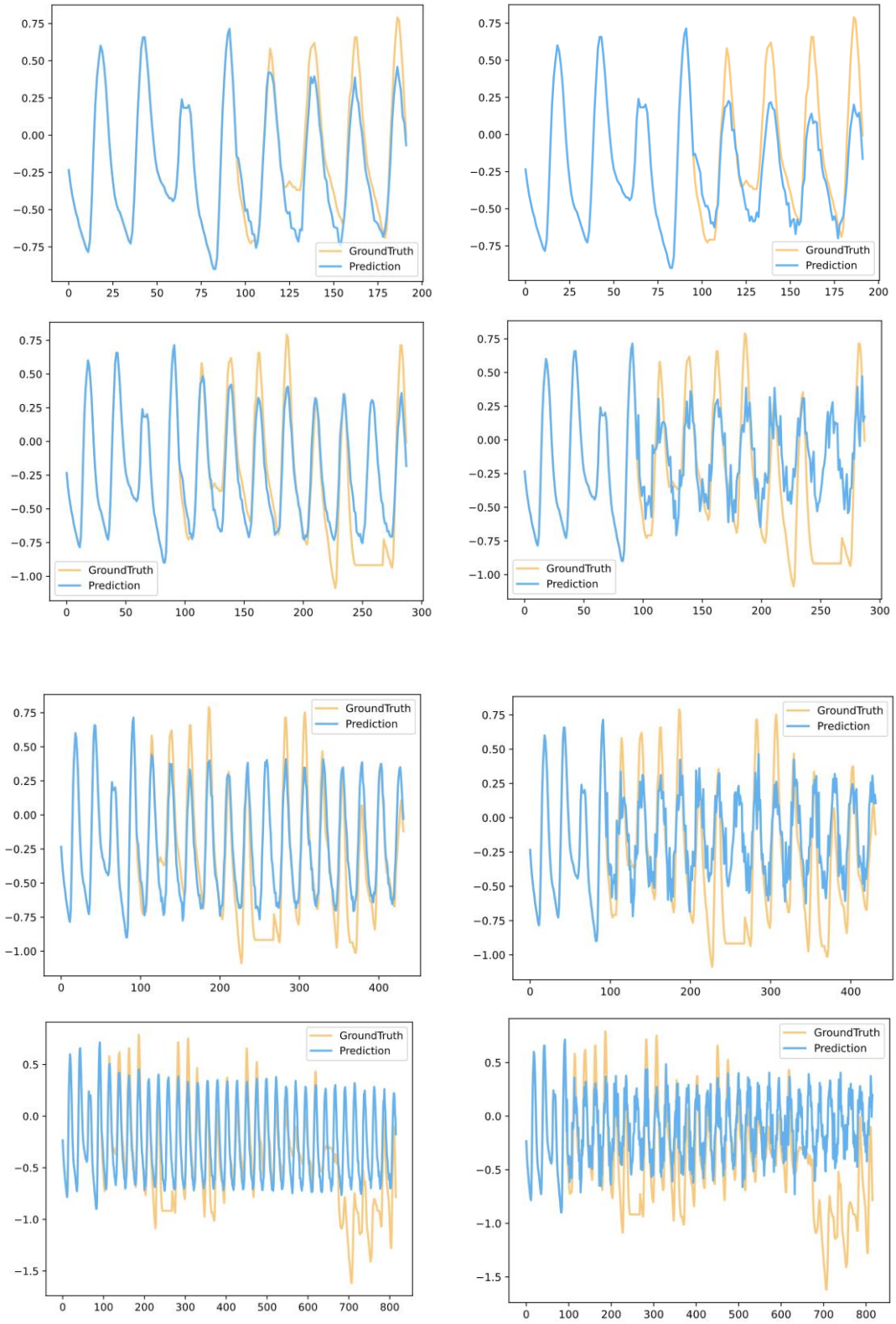


Figure 8. Comparison of prediction performance between CS4(left) and Rlinear(right) on the ETTh2 dataset across different prediction lengths.

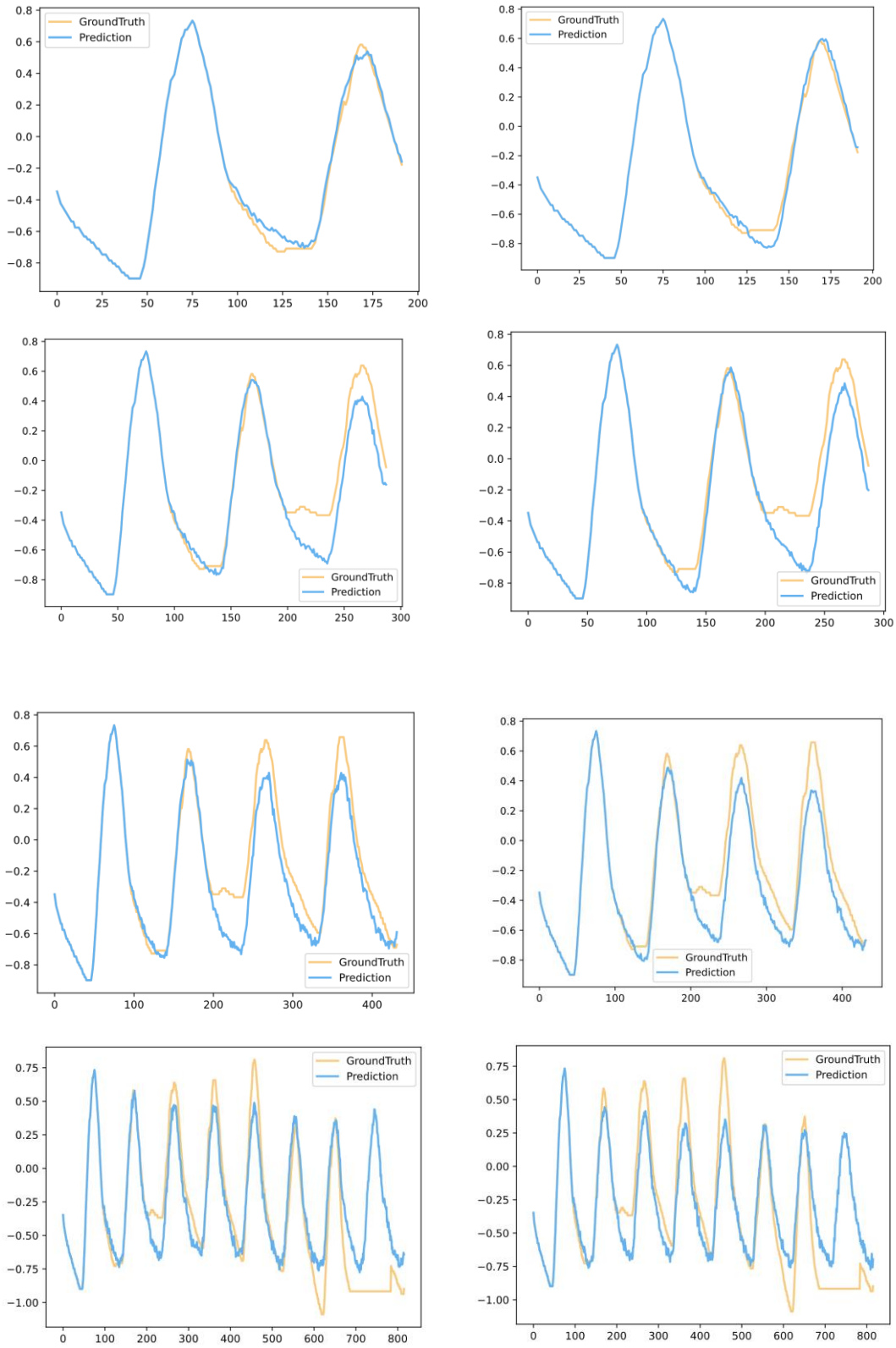


Figure 9 . Comparison of prediction performance between CS4(left) and PatchTST(right) on the ETTm2 dataset across different prediction lengths.

4.3.3 Computational Efficiency Analysis

To examine relative computational costs of CS4 versus the five comparison approaches, we selected the ETTh2 dataset where CS4 performed best in Table 2, and conducted comparisons under fixed configurations (pred_len=96, layer=2, d_model=256), analyzing training time and memory consumption of each model, as shown in Table 4.

Table 4. Computational efficiency comparison on the ETTh2 dataset (pred_len=96, layer=2, d_model=256).

Model	prediction error(MSE)	Training time per round (s)	GPU memory footprint(MB)	CPU memory footprint(MB)
CS4	0.281	2.91	290	590.5
iTransformer	0.297	2.94	202	540.8
PatchTST	0.302	3.58	264	549.6
TimesNet	0.340	4.85	347	610.8
RLinear	0.288	1.32	158	565.0
LSTM	0.796	2.15	180	525.0

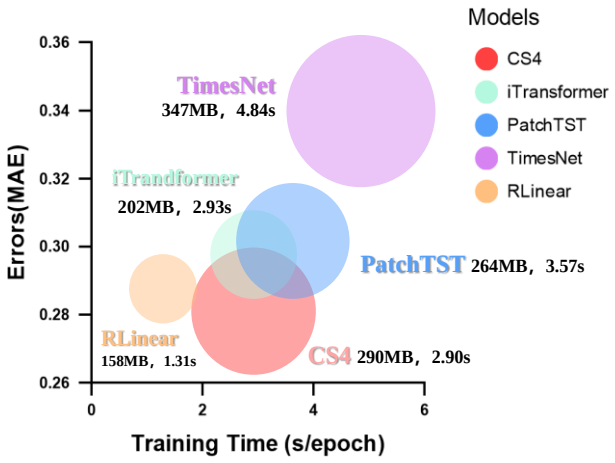


Figure 10. Bubble chart of computational efficiency versus prediction error for each model on the ETTh2 dataset.

Figure 10 visually demonstrates the trade-off relationship among prediction error, training time, and memory consumption for each model. Along the x-axis, training duration per cycle (seconds/epoch) is plotted, while the y-axis displays forecast deviation (MAE), with circle dimensions corresponding to GPU memory utilization (MB). The ideal model should be positioned in the lower-left region of the graph, indicating fast training speed, low prediction

error, and small memory footprint. The experimental findings reveal how various approaches balance processing resource requirements against forecasting accuracy. RLinear, as a linear model, demonstrates the fastest training speed (averaging only 1.32 seconds per epoch) and lowest GPU memory consumption (158MB), but its prediction error (MSE=0.288) is slightly higher than CS4. TimesNet incurs the highest computational cost, with training time per epoch (4.85 seconds) significantly longer than other models and the highest GPU memory consumption (347MB), yet its prediction performance is relatively weak.

Regarding training speed, CS4 demonstrates notable efficiency, requiring only 2.91 seconds per epoch—marginally faster than iTransformer (2.94 seconds) and substantially quicker than both PatchTST (3.58 seconds) and TimesNet (4.85 seconds). Although CS4's GPU memory consumption (290MB) is higher than iTransformer (202MB) and PatchTST (264MB), this moderate memory usage is reasonable considering it achieves the best prediction accuracy (MSE=0.281). From a CPU memory usage perspective, the differences among models are relatively minor, with CS4 (590.5MB) at a moderate level. Compared to the more structurally complex TimesNet, CS4 not only reduces prediction error by 17.4% but also improves training speed by 40% while decreasing GPU memory usage by approximately 16.4%, demonstrating superior computational efficiency.

5 Conclusion

This study introduced the Channel-aware Structured State Space Sequence model (CS4), a novel approach for modeling long time sequences that integrates both temporal and frequency domain information while maintaining computational efficiency. This study tackled fundamental obstacles related to identifying extended temporal patterns within sequential data, where existing methods face significant limitations: recurrent neural networks struggle with gradient instability, Transformers incur quadratic computational complexity, and normal state space models inadequately utilize frequency domain information.

The proposed CS4 model makes three principal contributions to the field. First, it shows potential in extending modeling capabilities for multi-channel features while preserving less than quadratic computational complexity through innovative use of instance normalization and channel rearrangement techniques. Second, the dual-branch architecture captures both temporal and spectral characteristics: the temporal branch adopts an improved S4D module with gating for dynamic dependencies, while the frequency branch leverages FFT and spectral attention to model periodic patterns. Finally, the multi-channel feature extraction and fusion strategy achieves effective inter-channel feature interactions across various stages of the model architecture, balancing channel independence and correlation.

Comprehensive experiments conducted on seven public datasets demonstrated the effectiveness of CS4, demonstrating superior results when evaluated against five leading benchmark approaches. Across comprehensive evaluations on five datasets, the CS4 model achieved first place in 11 out of 20 cases and second place in 3 cases, confirming its robustness and effectiveness across diverse forecasting scenarios. Particularly in long-sequence forecasting tasks, the temporal-frequency domain fusion architecture displayed clear advantages. The computational efficiency analysis confirmed that CS4 maintains an advantageous balance between prediction accuracy and resource utilization, making it suitable for practical applications in real-world scenarios.

Despite these promising results, several limitations should be acknowledged. The current implementation utilizes S4D, a diagonal variant of S4, sacrificing some modeling capacity for computational efficiency rather than employing the potentially more expressive but computationally intensive original S4 architecture. The research primarily focused on regularly sampled time series data, potentially limiting its applicability to irregular time series with varying sampling intervals or missing values. Furthermore, like many deep learning models,

CS4 lacks interpretability, making it challenging to provide clear explanations for specific predictions.

Future research directions should address these limitations and further advance S4 for time series forecasting. Extending the framework to handle irregularly sampled time series would significantly broaden its applicability in real-world scenarios. Exploring hierarchical modeling approaches could enhance scalability for ultra-high dimensional multivariate time series. Developing interpretable variants would increase utility in domains requiring explainable predictions, while incorporating adaptive learning mechanisms would improve robustness to concept drift in non-stationary environments. Integration with domain-specific physical constraints or prior knowledge could further enhance performance in specialized applications such as energy forecasting, weather prediction, or traffic modeling.

In summary, the CS4 model provides a promising approach for multi-channel long time series modeling by jointly leveraging temporal and frequency domain features under a computationally efficient framework. The study contributes additional confirmation that structured state space techniques represent feasible alternatives to classic recurrent and attention-focused architectures for sequential pattern recognition.

References

[1] Muresan S, Nakov P, Villavicencio A. Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*. Dublin, Ireland: ACL, 2022: 1234-1245.

[2] Liu Z, Liang C, Zheng G, Wei H. Fdti: Fine-grained deep traffic inference with roadnet-enriched graph. *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 2023: 174-191.

[3] Goller C, Kuchler A. Learning task-dependent distributed representations by backpropagation through structure. *Proceedings of International Conference on Neural Networks*. IEEE, 1996: 347-352.

[4] Hochreiter S, Schmidhuber J. Long short-term memory. *Neural Computation*, 1997, 9(8): 1735-1780.

[5] Huang X, Chan A, Li Y, et al. Natural Language Processing and Chinese Computing: 6th CCF International Conference, NLPCC 2017. *Proceedings of the 6th CCF International Conference*. Dalian, China: Springer, 2017: 45-67.

[6] Ma W, Smith J, Chen L, et al. ASTR: Transformer-based Alert-to-Stage Translator for multi-stage attack detection. *Neurocomputing (Amsterdam)*, 2025, 635: 130016.

[7] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need. *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., 2017: 5998-6008.

[8] Devlin J, Chang M, Lee K, et al. BERT: pre-training of deep bidirectional transformers for language understanding. *Proceedings of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*. Association for Computational Linguistics, 2019: 4171-4186.

[9] Brown T, Mann B, Ryder N, et al. Language models are few-shot learners. *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., 2020: 1877-1901.

[10] Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate. *Proceedings of the International Conference on Learning Representations (ICLR)*. ICLR, 2015: 1-15.

[11] Tay Y, Dehghani M, Bahri D, et al. Efficient transformers: A survey. *ACM Computing Surveys*, 2022, 55(6): 117:1-117:33.

[12] Gu A, Goel K, Ré C. Efficiently modeling long sequences with structured state spaces. *Proceedings of the International Conference on Learning Representations (ICLR)*. OpenReview, 2022: 1-15.

[13] Goel K. *Beyond Text: Applying Deep Learning to Signal Data*. 1st ed. PhD Thesis, Stanford University, 2024.

[14] Gu A, Goel K, Gupta A, et al. On the parameterization and initialization of diagonal state space models. *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., 2022: 9454-9467.

[15] Gupta A, Gu A, Berant J. Diagonal state spaces are as effective as structured state spaces. *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., 2022: 21234-21245.

[16] Gu A, Dao T, Ermon S, et al. HiPPO: Recurrent memory with optimal polynomial projections. *Proceedings of the Conference on Neural Information Processing Systems (NeurIPS)*. Curran Associates, Inc., 2020: 1474-1487.

[17] Cho K, Van Merriënboer B, Gulcehre C, et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint*, 2014. arXiv:1406.1078.

[18] Bengio Y, Simard P, Frasconi P. Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 1994, 5(2): 157-166.

[19] Bai S, Kolter J Z, Koltun V. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint*, 2018. arXiv:1803.01271.

[20] Li S, Jin X, Xuan Y, et al. Enhancing the locality and breaking the memory bottleneck of transformer on time series forecasting. *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2019: 5243-5253.

[21] Zhou H, Zhang S, Peng J, et al. Informer: Beyond efficient transformer for long sequence time-series forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press, 2021, 35(12): 11106-11115.

[22] Tay Y, Dehghani M, Bahri D, Metzler D. Efficient transformers: A survey. *arXiv preprint*, 2020. arXiv:2009.06732.

[23] Child R, Gray S, Radford A, Sutskever I. Generating long sequences with sparse transformers. *arXiv preprint*, 2019. arXiv:1904.10509.

[24] Katharopoulos A, Vyas A, Pappas N, Fleuret F. Transformers are RNNs: Fast autoregressive transformers with linear attention. *International Conference on Machine Learning*. PMLR, 2020: 5156-5165.

[25] Gu A, Dao T, Ermon S, Rudra A, Réa C. Combining recurrent, convolutional, and continuous-time models with linear state space layers. *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2021, 34: 332-342.

[26] Tay Y, Dehghani M, Abnar S, et al. Long range arena: A benchmark for efficient transformers. *International Conference on Learning Representations (ICLR)*. OpenReview, 2020: 1-15.

[27] Gu A, Goel K, Gupta A, et al. On the parameterization and initialization of diagonal state space models. *Advances in Neural Information Processing Systems*, 2022, 35: 35971-35983.

[28] Gupta S, Robertson J, Prorok A, Marecek J. Diagonal state spaces are as effective as structured state spaces. *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2022: 30123-30135.

[29] Gu A, Dao T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint*, 2023. arXiv:2312.00752.

[30] Wang Z, Zhou H, Wang S, et al. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. *International Conference on Machine Learning (ICML)*. PMLR, 2022: 22958-22973.

[31] Wu H, Xu T, Wang S, et al. TimesNet: Temporal 2D-variation modeling for general time series analysis. *International Conference on Learning Representations (ICLR)*. OpenReview, 2023: 1-15.

[32] Liu Y, Wu H, Wang J, Long M. SCINet: Time series modeling and forecasting with sample convolution and interaction. *arXiv preprint*, 2022. arXiv:2106.09305.

[33] Nie Y, Nam N H, Hoang C T, et al. A time series is worth 64 words: Long-term forecasting with transformers. *International Conference on Learning Representations (ICLR)*. OpenReview, 2023: 1-16.

[34] Liu S, Li W, Guan X, et al. CMamba: Efficient and accurate convolutional state space model for visual recognition. *arXiv preprint*, 2024. arXiv:2401.16866.

[35] Zhang R, Han Z, Liu J, Hu C. CTRW-SSM: Modeling irregularly-sampled time series with continuous-time random walks and state space models. *International Conference on Machine Learning (ICML)*. PMLR, 2023: 41250-41265.

[36] Pan X, Zhou F, Wang B, et al. STID-S4: Spatiotemporal interaction dynamics modeling with structured state space models for multivariate time series forecasting. *arXiv preprint*, 2023. arXiv:2309.05305.

[37] Mehta M, Gupta P, Ramasesh V V, et al. Hyena hierarchy: Towards larger convolutional language models. *International Conference on Machine Learning (ICML)*. PMLR, 2023: 24304-24319.

[38] Tang Y, Lu S, Gu A, et al. Long sequence time-series forecasting with state space models and ssm initialization. *arXiv preprint*, 2023. arXiv:2309.08598.

[39] Azeem A, Ismail I, Mohani S S, et al. Mitigating concept drift challenges in evolving smart grids: An adaptive ensemble LSTM for enhanced load forecasting. *Energy Reports*, 2025, 13: 1369-1383.

[40] Kantas N, Doucet A, Singh S S, et al. On particle methods for parameter estimation in state-space models. *Statistical Science*, 2015, 30(3): 328-351.

[41] Chen J. *Research on Compact Transformer Model Considering Long Distance Dependency*. PhD Thesis, University of Electronic Science and Technology, 2024. DOI:10.27005/d.cnki.gdzku.2024.000533.

[42] Lin Z. *A time series forecasting algorithm based on state space modeling*. Master's Thesis, Guangdong University of Technology, 2022. DOI:10.27029/d.cnki.ggdgu.2022.000947.

[43] Wang M, Gao Y, Li J. Image deblurring network based on Mamba and frequency domain fusion. *Computerized Measurement and Control*, 2025.

[44] Yu H, Fan J, Huang F. Construction of Convolutionally Enhanced Vision Mamba Model and Its Application. *Computer Technology and Development*, 2025.

[45] Peng J, Yang M, Zhang Q, et al. S4M: S4 for multivariate time series forecasting with Missing values. *arXiv preprint*, 2025. arXiv:2503.00900.

[46] Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate. *International Conference on Learning Representations (ICLR)*. OpenReview, 2015: 1-15.

[47] Lin Z, Feng M, Santos C N D, et al. A structured self-attentive sentence embedding. *International Conference on Learning Representations (ICLR)*. OpenReview, 2017: 1-15.

[48] Jozefowicz R, Zaremba W, Sutskever I. An empirical exploration of recurrent network architectures. *International Conference on Machine Learning*. PMLR, 2015: 2342-2350.

[49] Cooley J W, Tukey J W. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 1965, 19(90): 297-301.

[50] Brigham E O, Morrow R E. The fast Fourier transform. *IEEE Spectrum*, 1967, 4(12): 63-70.

[51] Duhamel P, Vetterli M. Fast Fourier transforms: A tutorial review and a state of the art. *Signal Processing*, 1990, 19(4): 259-299.

A. Appendix A: Description of the Baseline Model

We carefully selected 5 state-of-the-art models for our study. Their details are as follows:

1) **iTransformer** (Liu et al., 2023a) is an inverted Transformer-based model that captures cross channel dependencies via the self-attention mechanism and cross-time dependencies via linear projection. The source code is available at https://github.com/thuml/iTransformer.
2) **PatchTST** (Nie et al., 2022) is a patch-wise Transformer-based model that adopts a Channe Independent strategy. The source code is available at https://github.com/yuqinie98/PatchTST.
3) **TimesNet** (Wu et al., 2022) decomposes 1D time series into 2D time series based on multi periodicity and captures intra-period and inter-period correlations via convolution. The source code is available at https://github.com/thuml/Time-Series-Library.
4) **RLinear** (Li et al., 2023) is a Linear-based model with RevIN and a Channel-Independent strategy. The source code is available at https://github.com/plumprc/RTSF.
5) **LSTM** (Hochreiter & Schmidhuber, 1997) is a recurrent neural network architecture that employs memory cells with input, forget and output gates to capture long-term dependencies and mitigate the vanishing-gradient problem.

The modified frameworks of these models are shown in figure 11

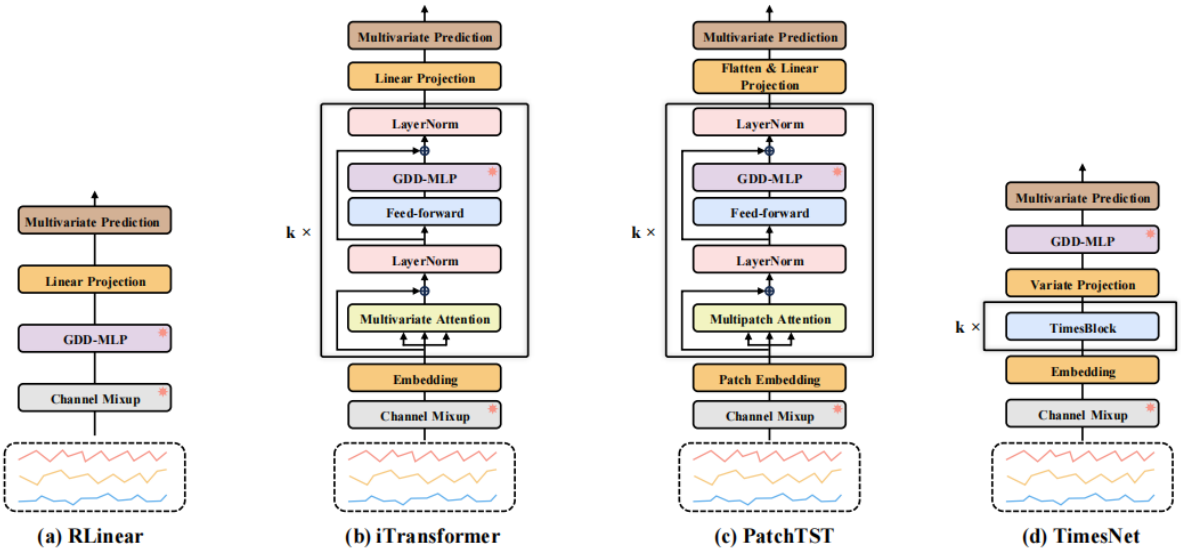


Figure 11. Modifications of the four chosen baselines. We retain the original architecture unchanged but apply Channel Mixup during training [40].

Appendix B: Parameter Sensitivity Analysis Details

Three complete tables integrating data on the effect of prediction length pred_len={96, 192, 336, 720}, number of layers layer={2, 3, 4} and model width d_model={128, 256} on prediction performance in Section 4.3.1:

Table 5. Impact of prediction length on the predictive performance of the CS4 model (% MSE growth rate with pred_len=96).

The raise of MSE is highlighted in red and the improvement in blue.

Layers & d_models	Pred _lens	ETTh1		ETTm1		ETTh2		ETTm2		ECL		Weather		Traffic	
		mse	%	mse	%	mse	%	mse	%	mse	%	mse	%	mse	%
2&128	96	0.387	—	0.331	—	0.285	—	0.177	—	0.194	—	0.195	—	0.625	—
	192	0.057	14.7	0.044	13.3	0.079	27.8	0.064	36.2	0.001	0.7	0.043	22.1	-0.032	-5.1
	336	0.103	26.6	0.076	23.1	0.124	43.4	0.124	70.0	0.016	8.2	0.095	48.8	-0.026	-4.2
	720	0.108	27.9	0.142	43.0	0.130	45.6	0.222	125.1	0.057	29.5	0.170	87.0	0.008	1.3
2&256	96	0.386	—	0.324	—	0.289	—	0.176	—	0.193	—	0.192	—	0.616	—
	192	0.057	14.8	0.050	15.5	0.071	24.5	0.064	36.5	0.002	0.8	0.043	22.5	-0.026	-4.2
	336	0.104	26.9	0.086	26.5	0.124	42.9	0.124	70.5	0.016	8.2	0.095	49.7	-0.020	-3.2
	720	0.095	24.6	0.148	45.8	0.124	43.0	0.223	126.5	0.057	29.6	0.171	89.0	0.012	1.9
3&128	96	0.387	—	0.325	—	0.285	—	0.180	—	0.194	—	0.192	—	0.622	—
	192	0.057	14.7	0.048	14.9	0.077	27.0	0.061	33.6	0.002	0.8	0.043	22.5	-0.031	-4.9
	336	0.104	26.9	0.081	24.9	0.122	42.8	0.121	67.3	0.016	8.4	0.096	50.0	-0.025	-3.9
	720	0.102	26.4	0.148	45.5	0.138	48.4	0.221	122.8	0.058	29.7	0.171	89.3	0.009	1.5
3&256	96	0.386	—	0.324	—	0.282	—	0.176	—	0.192	—	0.190	—	0.607	—
	192	0.059	15.3	0.051	15.6	0.078	27.6	0.065	36.8	0.002	0.9	0.043	22.7	-0.021	-3.5
	336	0.106	27.5	0.081	25.1	0.126	44.8	0.125	71.1	0.016	8.3	0.096	50.3	-0.015	-2.4
	720	0.121	31.3	0.149	46.0	0.128	45.5	0.224	127.2	0.057	29.8	0.171	89.8	0.018	2.9
4&128	96	0.387	—	0.325	—	0.284	—	0.177	—	0.194	—	0.190	—	0.618	—
	192	0.057	14.7	0.049	15.1	0.076	26.8	0.064	36.5	0.001	0.8	0.044	23.3	-0.028	-4.5
	336	0.104	26.9	0.083	25.5	0.122	42.9	0.125	71.0	0.016	8.3	0.096	50.7	-0.021	-3.5
	720	0.107	27.6	0.152	46.6	0.134	47.1	0.225	127.4	0.057	29.7	0.172	90.4	0.013	2.0
4&256	96	0.386	—	0.324	—	0.281	—	0.175	—	0.192	—	0.189	—	0.602	—
	192	0.057	14.8	0.049	15.1	0.087	30.9	0.066	37.4	0.002	1.0	0.044	23.2	-0.018	-3.1
	336	0.105	27.2	0.081	24.9	0.122	43.3	0.126	71.9	0.016	8.5	0.096	50.8	-0.012	-2.0
	720	0.108	28.0	0.148	45.7	0.129	45.9	0.224	128.0	0.057	29.9	0.171	90.3	0.021	3.4

Table 6. Impact of the number of layers on the predictive performance of the CS4 model (percent change in MSE with layer=2).

The raise of MSE is highlighted in red and the improvement in blue.

Pred_len & d_models	layers	ETTh1		ETTm1		ETTh2		ETTm2		ECL		Weather		Traffic	
		mse	%	mse	%	mse	%	mse	%	mse	%	mse	%	mse	%
96&128	2	0.387	—	0.331	—	0.285	—	0.177	—	0.194	—	0.195	—	0.625	—
	3	-0.0003	-0.0816	-0.005	-1.60	-0.0004	-0.142	0.003	1.73	-0.0003	-0.176	-0.003	-1.63	-0.003	-0.403
	4	0.001	0.169	-0.005	-1.66	-0.001	-0.297	-0.001	-0.373	-0.0003	-0.162	-0.005	-2.65	-0.007	-1.08
96&256	2	0.386	—	0.324	—	0.289	—	0.176	—	0.193	—	0.192	—	0.616	—
	3	-0.0002	-0.0538	-0.0004	-0.127	-0.007	-2.26	-0.0001	-0.0656	-0.0003	-0.141	-0.002	-1.03	-0.008	-1.36
	4	-0.0005	-0.119	0.0001	0.0419	-0.008	-2.61	-0.001	-0.417	-0.001	-0.311	-0.003	-1.42	-0.014	-2.27
192&128	2	0.444	—	0.374	—	0.365	—	0.242	—	0.195	—	0.238	—	0.593	—
	3	-0.0001	-0.0113	-0.001	-0.203	-0.003	-0.761	-0.001	-0.214	-0.0002	-0.114	-0.003	-1.36	-0.001	-0.237
	4	0.0002	0.0404	-0.0002	-0.0570	-0.004	-1.12	-0.001	-0.211	-0.0003	-0.153	-0.004	-1.71	-0.003	-0.448
192&256	2	0.443	—	0.374	—	0.360	—	0.240	—	0.194	—	0.235	—	0.590	—
	3	0.002	0.417	-0.00003	-0.00808	0.001	0.192	0.0003	0.127	-0.0001	-0.0529	-0.002	-0.840	-0.004	-0.676
	4	-0.001	-0.140	-0.001	-0.252	0.009	2.38	0.001	0.228	-0.0002	-0.106	-0.002	-0.905	-0.007	-1.15
336&128	2	0.490	—	0.407	—	0.409	—	0.301	—	0.210	—	0.290	—	0.599	—
	3	0.001	0.208	-0.001	-0.125	-0.002	-0.525	0.0005	0.161	-0.0001	-0.0498	-0.002	-0.845	-0.001	-0.173
	4	0.002	0.313	0.001	0.309	-0.003	-0.632	0.001	0.226	-0.0002	-0.116	-0.004	-1.36	-0.002	-0.352
336&256	2	0.490	—	0.410	—	0.413	—	0.300	—	0.209	—	0.287	—	0.596	—
	3	0.002	0.424	-0.005	-1.24	-0.004	-0.923	0.001	0.276	-0.0001	-0.0507	-0.002	-0.628	-0.003	-0.577
	4	0.001	0.279	-0.005	-1.24	-0.010	-2.31	0.001	0.387	-0.0002	-0.0938	-0.002	-0.674	-0.006	-1.01
720&128	2	0.495	—	0.473	—	0.416	—	0.399	—	0.251	—	0.365	—	0.633	—
	3	-0.006	-1.19	0.001	0.123	0.007	1.74	0.003	0.669	0.00002	0.00612	-0.002	-0.424	-0.002	-0.240
	4	-0.002	-0.329	0.004	0.849	0.003	0.726	0.003	0.645	-0.0001	-0.0236	-0.003	-0.856	-0.002	-0.357
720&256	2	0.481	—	0.472	—	0.413	—	0.399	—	0.250	—	0.363	—	0.628	—
	3	0.026	5.33	0.0002	0.0520	-0.002	-0.544	0.001	0.220	-0.0001	-0.0275	-0.002	-0.566	-0.003	-0.430
	4	0.012	2.56	0.0001	0.0139	-0.003	-0.629	0.001	0.215	-0.0002	-0.0954	-0.003	-0.749	-0.005	-0.844

Table 7. Impact of model width on CS4 model prediction performance (percent change in MSE from 128 to 256 for d_model).

The raise of MSE is highlighted in red and the improvement in blue.

Pred_len & layers	d_models	ETTh1		ETTm1		ETTh2		ETTm2		ECL		Weather		Traffic	
		mse	%	mse	%	mse	%	mse	%	mse	%	mse	%	mse	%
96&2	128	0.38683	—	0.33058	—	0.28534	—	0.17730	—	0.19402	—	0.19511	—	0.62475	—
	256	-0.00085	-0.221	-0.00656	-1.985	0.00348	1.219	-0.00116	-0.653	-0.00130	-0.668	-0.00311	-1.594	-0.00906	-1.450
96&3	128	0.38652	—	0.32528	—	0.28494	—	0.18036	—	0.19368	—	0.19193	—	0.62223	—
	256	-0.00075	-0.193	-0.00168	-0.515	-0.00264	-0.925	-0.00433	-2.402	-0.00122	-0.632	-0.00191	-0.993	-0.01490	-2.395
96&4	128	0.38749	—	0.32510	—	0.28450	—	0.17664	—	0.19370	—	0.18994	—	0.61803	—
	256	-0.00197	-0.507	-0.00095	-0.291	-0.00322	-1.133	-0.00123	-0.697	-0.00158	-0.816	-0.00068	-0.358	-0.01632	-2.640
192&2	128	0.44383	—	0.37446	—	0.36472	—	0.24156	—	0.19546	—	0.23827	—	0.59289	—
	256	-0.00041	-0.093	-0.00027	-0.073	-0.00510	-1.399	-0.00108	-0.445	-0.00124	-0.635	-0.00305	-1.282	-0.00289	-0.488
192&3	128	0.44378	—	0.37370	—	0.36194	—	0.24105	—	0.19524	—	0.23503	—	0.59149	—

	256	0.00148	0.334	0.00045	0.122	-0.00164	-0.453	-0.00025	-0.105	-0.00112	-0.574	-0.00179	-0.761	-0.00548	-0.927
192&4	128	0.44401	—	0.37425	—	0.36063	—	0.24105	—	0.19517	—	0.23420	—	0.59024	—
	256	-0.00121	-0.273	-0.00100	-0.268	0.00753	2.088	-0.00002	-0.007	-0.00115	-0.588	-0.00111	-0.473	-0.00700	-1.185
336&2	128	0.48959	—	0.40690	—	0.40918	—	0.30133	—	0.20998	—	0.29028	—	0.59875	—
	256	0.00005	0.010	0.00294	0.721	0.00341	0.835	-0.00096	-0.317	-0.00138	-0.656	-0.00290	-0.997	-0.00283	-0.472
336&3	128	0.49061	—	0.40640	—	0.40703	—	0.30182	—	0.20988	—	0.28782	—	0.59771	—
	256	0.00110	0.225	-0.00166	-0.408	0.00176	0.432	-0.00061	-0.203	-0.00138	-0.657	-0.00225	-0.780	-0.00523	-0.875
336&4	128	0.49112	—	0.40816	—	0.40659	—	0.30202	—	0.20974	—	0.28634	—	0.59664	—
	256	-0.00012	-0.024	-0.00341	-0.837	-0.00355	-0.872	-0.00047	-0.157	-0.00133	-0.634	-0.00090	-0.313	-0.00677	-1.134
720&2	128	0.49531	—	0.47269	—	0.41554	—	0.39911	—	0.25123	—	0.36479	—	0.63286	—
	256	-0.01397	-2.821	-0.00038	-0.080	-0.00251	-0.604	-0.00006	-0.015	-0.00141	-0.561	-0.00199	-0.545	-0.00522	-0.825
720&3	128	0.48941	—	0.47327	—	0.42278	—	0.40178	—	0.25125	—	0.36325	—	0.63133	—
	256	0.01760	3.597	-0.00071	-0.151	-0.01199	-2.837	-0.00185	-0.462	-0.00149	-0.595	-0.00250	-0.687	-0.00640	-1.014
720&4	128	0.49368	—	0.47670	—	0.41856	—	0.40168	—	0.25117	—	0.36167	—	0.63060	—
	256	-0.00002	-0.005	-0.00432	-0.907	-0.00813	-1.941	-0.00178	-0.443	-0.00159	-0.633	-0.00158	-0.438	-0.00826	-1.310

Appendix C: Complete Experimental Results

The detailed experimental results for the seven datasets are organized as follows:

1) Tables 8 to 14: Complete experimental results of the CS4 model on the seven datasets (sub-tabulated by dataset)

2) Table 15: Comparison of the performance of the CS4 with the five baseline models on the five datasets (MSE and MAE)

3) Table 16: Comparison of computational efficiency between the CS4 and the five baseline models on the ETTh2 dataset

Table 8. Complete experimental results of the CS4 model on the ETTh1 dataset.

Dataset	pred_lens	layers	d_models	MSE	MAE
ETTh1	96	2	128	0.386833102	0.388819754
ETTh1	96	2	256	0.385979891	0.387140393
ETTh1	96	3	128	0.386517256	0.389093965
ETTh1	96	3	256	0.385772049	0.387194306
ETTh1	96	4	128	0.387486577	0.388782620
ETTh1	96	4	256	0.385521054	0.387149364
ETTh1	192	2	128	0.443831623	0.419757456
ETTh1	192	2	256	0.443417072	0.418291569
ETTh1	192	3	128	0.443781435	0.419740260
ETTh1	192	3	256	0.445264548	0.421625704
ETTh1	192	4	128	0.444010824	0.419959843

ETTh1	192	4	256	0.442798316	0.418283910
ETTh1	336	2	128	0.489588141	0.441198438
ETTh1	336	2	256	0.489636302	0.440308630
ETTh1	336	3	128	0.490608752	0.441379875
ETTh1	336	3	256	0.491713583	0.442447543
ETTh1	336	4	128	0.491121173	0.441585302
ETTh1	336	4	256	0.491001457	0.442593902
ETTh1	720	2	128	0.495310128	0.463008285
ETTh1	720	2	256	0.481337637	0.460873306
ETTh1	720	3	128	0.489411354	0.461201906
ETTh1	720	3	256	0.507015347	0.469919682
ETTh1	720	4	128	0.493680894	0.462561846
ETTh1	720	4	256	0.493657142	0.466105938

Table 9. Complete experimental results of the CS4 model on the ETTm1 dataset.

Dataset	pred_lens	layers	d_models	MSE	MAE
ETTm1	96	2	128	0.330580115	0.350060642
ETTm1	96	2	256	0.324017525	0.346868694
ETTm1	96	3	128	0.325282931	0.347024381
ETTm1	96	3	256	0.323607266	0.346034467
ETTm1	96	4	128	0.325100869	0.346891552
ETTm1	96	4	256	0.324153423	0.345749646
ETTm1	192	2	128	0.374464422	0.371219367
ETTm1	192	2	256	0.374189436	0.371330321
ETTm1	192	3	128	0.373704970	0.37065509
ETTm1	192	3	256	0.374159187	0.370276749
ETTm1	192	4	128	0.374251127	0.370660841
ETTm1	192	4	256	0.373246372	0.369973242
ETTm1	336	2	128	0.406903177	0.393099636
ETTm1	336	2	256	0.409838200	0.395695418
ETTm1	336	3	128	0.406395882	0.393054366
ETTm1	336	3	256	0.404739708	0.391867369
ETTm1	336	4	128	0.408160120	0.394061834
ETTm1	336	4	256	0.404745609	0.392022848
ETTm1	720	2	128	0.472690642	0.429889619
ETTm1	720	2	256	0.472313911	0.429609448

ETTm1	720	3	128	0.473272234	0.430398941
ETTm1	720	3	256	0.472559571	0.429741591
ETTm1	720	4	128	0.476704031	0.432067037
ETTm1	720	4	256	0.472379476	0.429766059

Table 10. Complete experimental results of the CS4 model on the ETTh2 dataset.

Dataset	pred_lens	layers	d_models	MSE	MAE
ETTh2	96	2	128	0.285344005	0.331499934
ETTh2	96	2	256	0.288820982	0.337063998
ETTh2	96	3	128	0.284937769	0.331422508
ETTh2	96	3	256	0.282301873	0.330052763
ETTh2	96	4	128	0.284497350	0.330994129
ETTh2	96	4	256	0.281274706	0.329285324
ETTh2	192	2	128	0.364718169	0.382042170
ETTh2	192	2	256	0.359615147	0.379573733
ETTh2	192	3	128	0.361943573	0.381015331
ETTh2	192	3	256	0.360303819	0.380371332
ETTh2	192	4	128	0.360629767	0.379954904
ETTh2	192	4	256	0.368158013	0.384691298
ETTh2	336	2	128	0.409180373	0.417880625
ETTh2	336	2	256	0.412595183	0.422058254
ETTh2	336	3	128	0.407030970	0.416708648
ETTh2	336	3	256	0.408788413	0.416260362
ETTh2	336	4	128	0.406594515	0.415559798
ETTh2	336	4	256	0.403047264	0.414620757
ETTh2	720	2	128	0.415544003	0.434006423
ETTh2	720	2	256	0.413033038	0.431859970
ETTh2	720	3	128	0.422781736	0.437921613
ETTh2	720	3	256	0.410787404	0.430417359
ETTh2	720	4	128	0.418558925	0.435958803
ETTh2	720	4	256	0.410433561	0.430832744

Table 11. Complete experimental results of the CS4 model on the ETTm2 dataset.

Dataset	pred_lens	layers	d_models	MSE	MAE
ETTm2	96	2	128	0.177303031	0.255913138
ETTm2	96	2	256	0.176145419	0.254847407
ETTm2	96	3	128	0.180363104	0.257228017
ETTm2	96	3	256	0.176029921	0.254868776
ETTm2	96	4	128	0.176642239	0.255889833
ETTm2	96	4	256	0.175410762	0.254193038
ETTm2	192	2	128	0.241564155	0.297768950
ETTm2	192	2	256	0.240488857	0.297305822
ETTm2	192	3	128	0.241047725	0.297854125
ETTm2	192	3	256	0.240795061	0.297664762
ETTm2	192	4	128	0.241054550	0.297760308
ETTm2	192	4	256	0.241037935	0.297822654
ETTm2	336	2	128	0.301334083	0.335390866
ETTm2	336	2	256	0.300378561	0.335746974
ETTm2	336	3	128	0.301820278	0.335942715
ETTm2	336	3	256	0.301206678	0.336545974
ETTm2	336	4	128	0.302015811	0.337043881
ETTm2	336	4	256	0.301542252	0.336495221
ETTm2	720	2	128	0.399110347	0.392447889
ETTm2	720	2	256	0.399048746	0.393538326
ETTm2	720	3	128	0.401779473	0.392633587
ETTm2	720	3	256	0.399925172	0.394458711
ETTm2	720	4	128	0.401683301	0.393144786
ETTm2	720	4	256	0.399905026	0.392449051

Table 12. Complete experimental results of the CS4 model on the Electricity dataset.

Dataset	pred_lens	layers	d_models	MSE	MAE
Electricity	96	2	128	0.194017038	0.266954035
Electricity	96	2	256	0.192721784	0.265145808
Electricity	96	3	128	0.193675399	0.266602576
Electricity	96	3	256	0.192450672	0.264845550
Electricity	96	4	128	0.193702817	0.266644865
Electricity	96	4	256	0.192122638	0.264552027
Electricity	192	2	128	0.195464626	0.271736711

Electricity	192	2	256	0.194224179	0.269719481
Electricity	192	3	128	0.195242614	0.271408021
Electricity	192	3	256	0.194121391	0.269613117
Electricity	192	4	128	0.195165843	0.271307081
Electricity	192	4	256	0.194018424	0.269467831
Electricity	336	2	128	0.209981456	0.287242651
Electricity	336	2	256	0.208604410	0.285073340
Electricity	336	3	128	0.209876865	0.287103564
Electricity	336	3	256	0.208498701	0.284948379
Electricity	336	4	128	0.209738106	0.286934495
Electricity	336	4	256	0.208408773	0.284802228
Electricity	720	2	128	0.251230568	0.319988698
Electricity	720	2	256	0.249820173	0.317936748
Electricity	720	3	128	0.251245946	0.319980770
Electricity	720	3	256	0.249751404	0.317831993
Electricity	720	4	128	0.251171201	0.319868922
Electricity	720	4	256	0.249581829	0.317665815

Table 13. Complete experimental results of the CS4 model on the Weather dataset.

Dataset	pred_lens	layers	d_models	MSE	MAE
Weather	96	2	128	0.195105687	0.223650515
Weather	96	2	256	0.191996589	0.222196221
Weather	96	3	128	0.191926077	0.222464472
Weather	96	3	256	0.190020606	0.221710324
Weather	96	4	128	0.189944670	0.221260816
Weather	96	4	256	0.189265266	0.221060485
Weather	192	2	128	0.238272265	0.260201931
Weather	192	2	256	0.235217288	0.258542925
Weather	192	3	128	0.235028282	0.258765906
Weather	192	3	256	0.233240426	0.258341551
Weather	192	4	128	0.234197348	0.258730590
Weather	192	4	256	0.233089596	0.258203566
Weather	336	2	128	0.290276378	0.297462881
Weather	336	2	256	0.287381232	0.296192497
Weather	336	3	128	0.287823081	0.296687573
Weather	336	3	256	0.285577744	0.296032488

Weather	336	4	128	0.286339998	0.296283931
Weather	336	4	256	0.285443693	0.295930982
Weather	720	2	128	0.364793539	0.345938236
Weather	720	2	256	0.362805903	0.345041931
Weather	720	3	128	0.363247454	0.345438510
Weather	720	3	256	0.360752016	0.344492286
Weather	720	4	128	0.361671925	0.344986171
Weather	720	4	256	0.360088974	0.344634384

Table 14. Complete experimental results of the CS4 model on the Traffic dataset.

Dataset	pred_lens	layers	d_models	MSE	MAE
Traffic	96	2	128	0.624746084	0.356005102
Traffic	96	2	256	0.615688980	0.348037153
Traffic	96	3	128	0.622228801	0.355185211
Traffic	96	3	256	0.607324243	0.345228583
Traffic	96	4	128	0.618028343	0.353727669
Traffic	96	4	256	0.601711631	0.343628705
Traffic	192	2	128	0.592893481	0.337858945
Traffic	192	2	256	0.589999914	0.33130604
Traffic	192	3	128	0.591490746	0.337467641
Traffic	192	3	256	0.586008847	0.330001324
Traffic	192	4	128	0.590239525	0.336966246
Traffic	192	4	256	0.583243608	0.329314351
Traffic	336	2	128	0.598748446	0.340769082
Traffic	336	2	256	0.595920324	0.334370524
Traffic	336	3	128	0.597713232	0.340444118
Traffic	336	3	256	0.592482448	0.333259761
Traffic	336	4	128	0.596641421	0.340177000
Traffic	336	4	256	0.589874685	0.332495421
Traffic	720	2	128	0.632855952	0.359649003
Traffic	720	2	256	0.627633929	0.352867305
Traffic	720	3	128	0.631334066	0.359127730
Traffic	720	3	256	0.624934018	0.351848483
Traffic	720	4	128	0.630598128	0.359001964
Traffic	720	4	256	0.622335613	0.350993931

Table 15. Performance of CS4 vs. baseline model on five datasets.

The best is highlighted in red and the runner-up in **blue.**

Models	CS4		iTransformer		PatchTST		TimesNet		RLinear		LSTM	
Metric	mse	mae	mse	mae	mse	mae	mse	mae	mse	mae	mse	mae
ETTm1	**0.394**	**0.384**	0.407	0.410	**0.387**	**0.400**	0.400	0.406	0.414	0.408	0.776	0.635
ETTm2	**0.279**	**0.320**	0.288	0.332	**0.281**	**0.326**	0.291	0.333	0.286	0.327	0.471	0.449
ETTh1	**0.453**	**0.429**	0.454	0.447	0.469	0.454	0.485	0.450	**0.446**	**0.434**	0.941	0.707
ETTh2	**0.366**	**0.390**	0.383	0.407	0.387	0.407	0.414	0.427	**0.374**	**0.398**	0.785	0.601
ECL	0.211	**0.284**	**0.178**	**0.270**	0.216	0.304	**0.192**	0.295	0.219	0.298	0.375	0.432

Table 16. Computational efficiency of CS4 vs. baseline model on ETTh2 dataset.

Models		CS4	iTransformer	PatchTST	TimesNet	RLinear
MSE		0.281	0.297	0.302	0.340	0.288
Time/epoch (s)	1	3.10313010	3.30661726	3.90021276	4.83572602	1.54861950
	2	2.88417315	2.872093916	3.54202556	4.81239510	1.25663065
	3	2.90356946	2.863516569	3.51383304	4.87926458	1.27665686
	4	2.81575465	2.906806946	3.50708556	4.82311765	1.31343698
	5	2.84230876	2.891920328	3.53779768	4.86193827	1.28493571
	6	2.88537645	2.891920328	3.51348376	4.84732019	1.38257670
	7	—	2.825380325	3.51784849	4.83270193	1.30732965
	8	—	—	3.60671806	4.87250931	1.24468827
	9	—	—	—	—	1.26913642
	10	—	—	—	—	1.26850819
	Avg	2.90571876	2.93689366	3.57987561	4.84562163	1.31525189
Maximum GPU (MB)		290.00	202.00	264.00	347.00	158.00
Average GPU (MB)		246.00	177.06	232.63	301.73	144.43
Maximum CPU (MB)		590.50	540.83	549.63	610.77	567.96
Average CPU (MB)		590.50	540.83	549.63	610.77	567.96

Appendix D: Showcase

As shown in figure 12, figure 13, figure 14, figure 15, and figure 16, we visualize the channel results of five datasets under CS4.

As shown in figure 17, figure 18, figure 19, figure 20, and figure 21, we visualize the forecasting results of other five datasets (excpt for ETTh2 and ETTm2 in Section 4.3.2) under CS4.

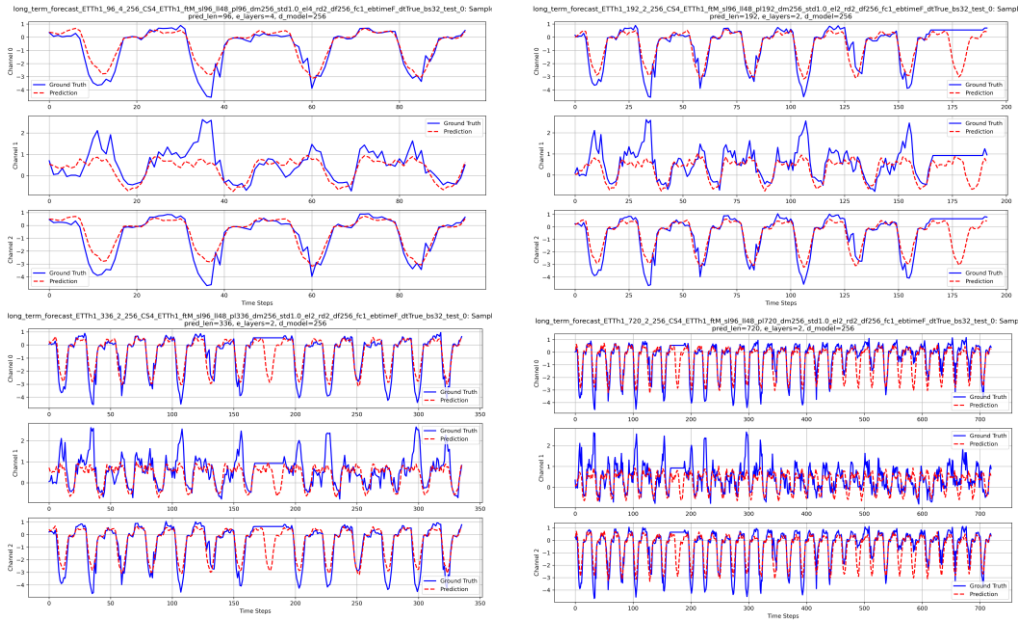


Figure 12. Multi-channel time series prediction results of CS4 model under ETTh1 dataset.

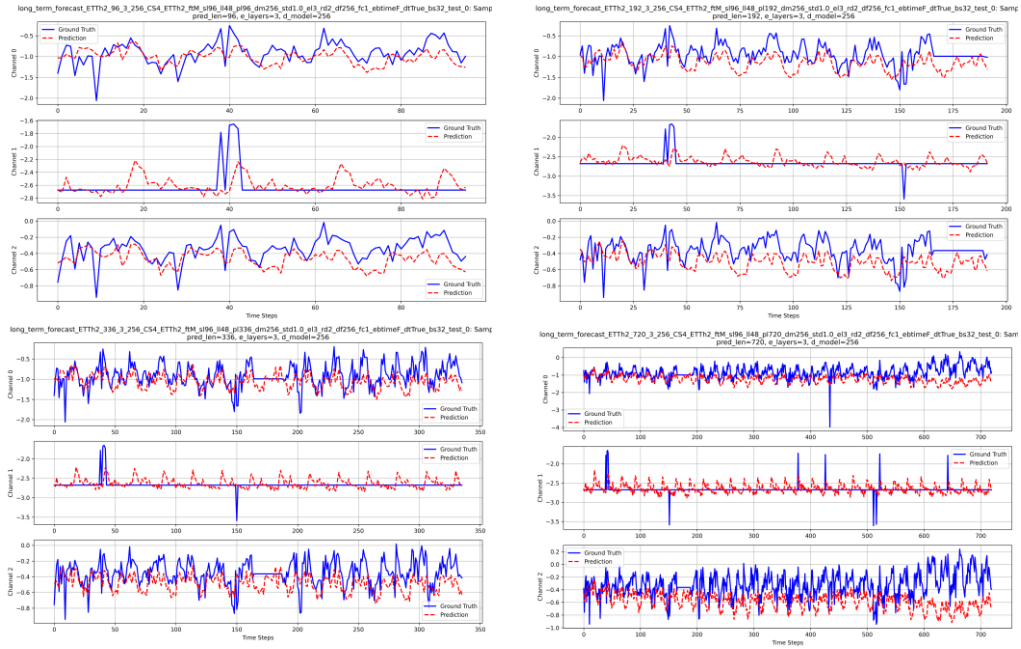


Figure 13. Multi-channel time series prediction results of CS4 model under ETTh2 dataset.

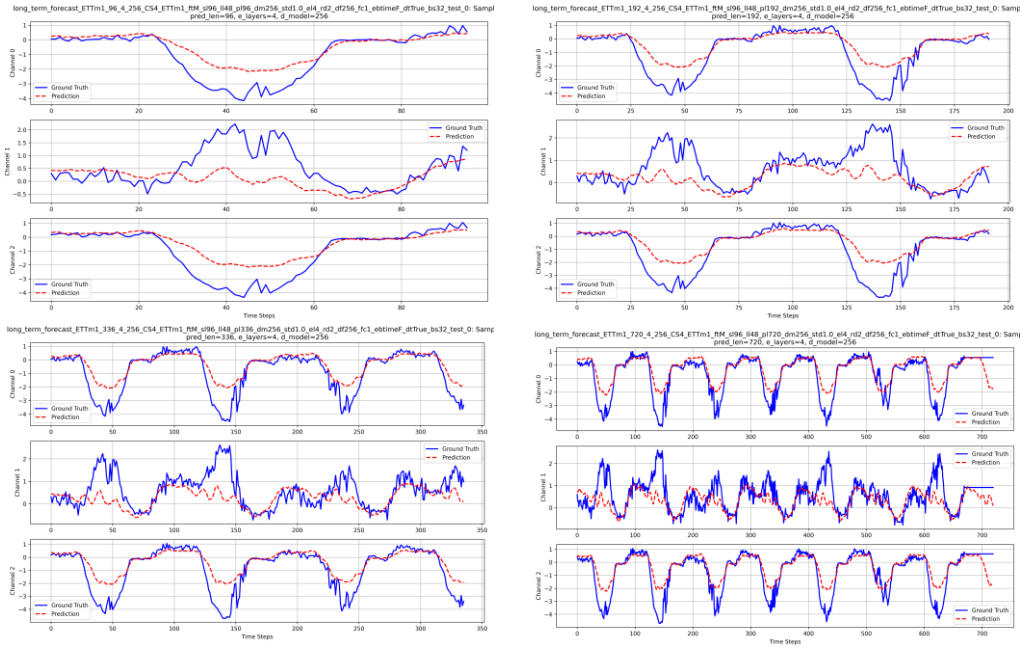


Figure 14. Multi-channel time series prediction results of CS4 model under ETTm1 dataset.



Figure 15. Multi-channel time series prediction results of CS4 model under ETTm2 dataset.

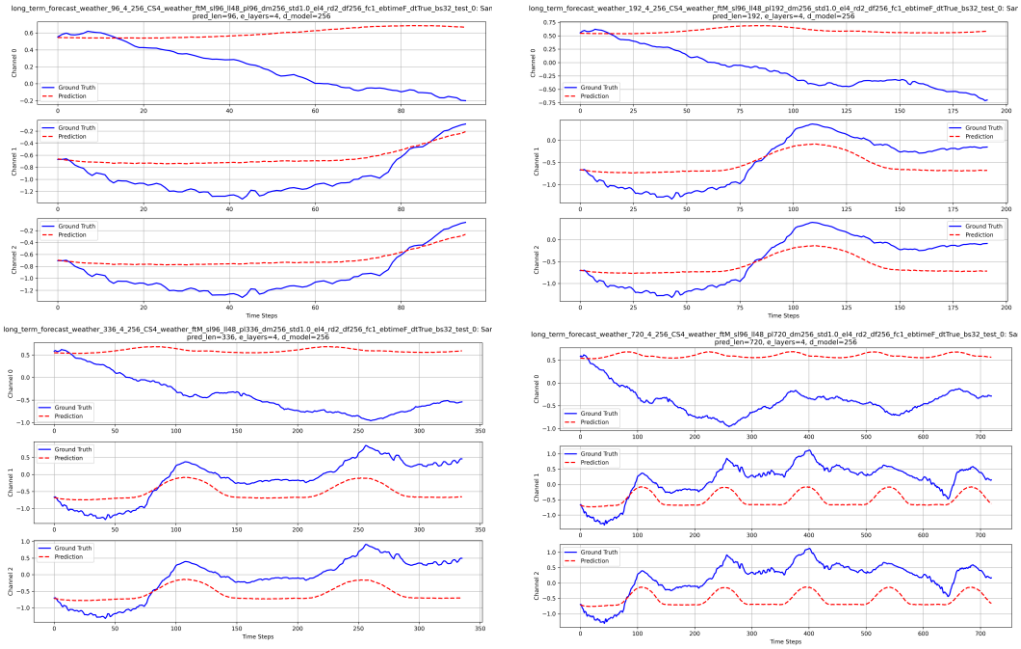


Figure 16. Multi-channel time series prediction results of CS4 model under Weather dataset.

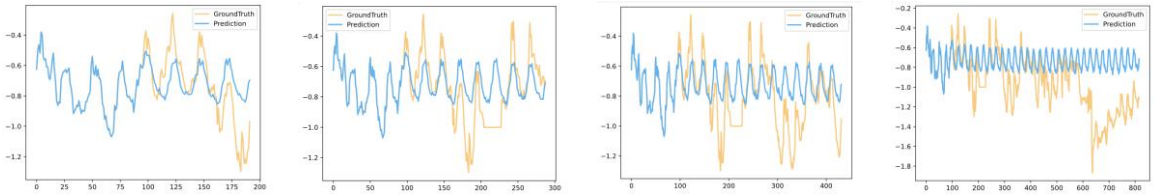


Figure 17. Prediction cases for ETTh1 under CS4.

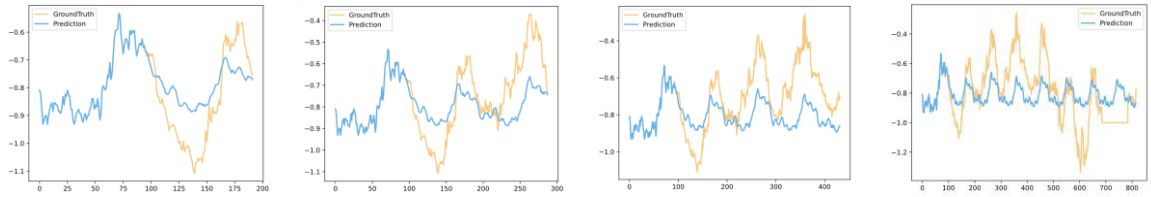


Figure 18. Prediction cases for ETTm1 under CS4.

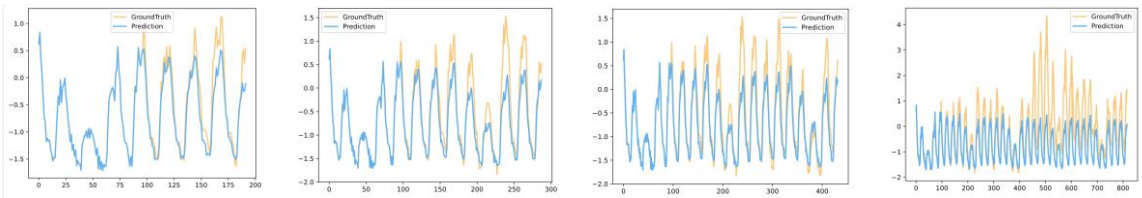


Figure 19. Prediction cases for Electricity under CS4.

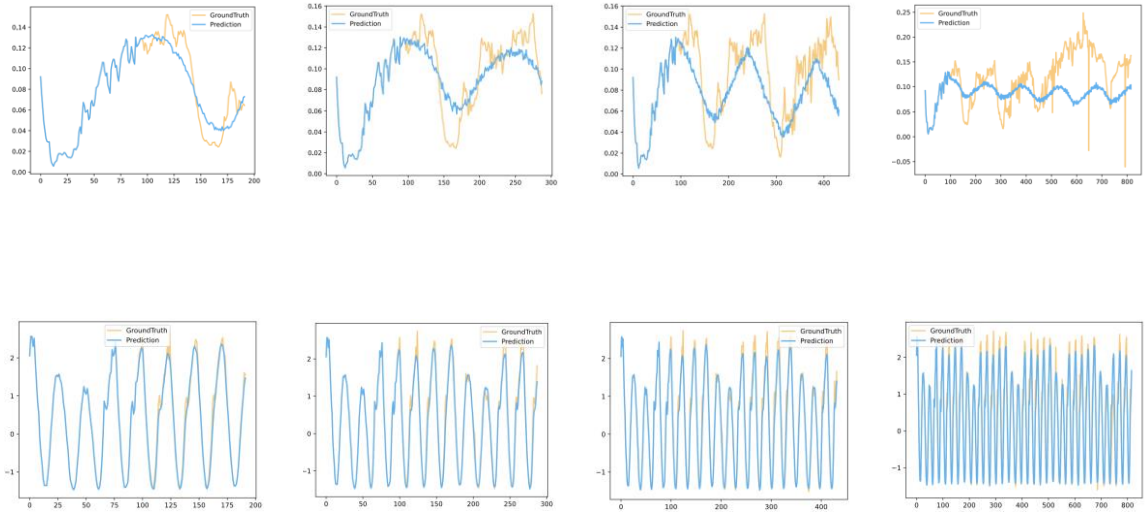


Figure 20. Prediction cases for Weather and Traffic under CS4.

B University's Plagiarism Statement

This statement is reviewed annually, and the definitive statement is in the University Calendar.

The University's degrees and other academic awards are given in recognition of a student's personal achievement. All work submitted by students for assessment is accepted on the understanding that it is the student's own effort.

Plagiarism is defined as the submission or presentation of work, in any form, which is not one's own, without acknowledgement of the sources. Plagiarism includes inappropriate collaboration with others. Special cases of plagiarism can arise from a student using his or her own previous work (termed auto-plagiarism or self-plagiarism). Auto plagiarism includes using work that has already been submitted for assessment at this University or for any other academic award.

The incorporation of material without formal and proper acknowledgement (even with no deliberate intent to cheat) can constitute plagiarism. Work may be considered to be plagiarised if it consists of:

- a direct quotation;
- a close paraphrase;
- an unacknowledged summary of a source;
- direct copying or transcription.

With regard to essays, reports and dissertations, the rule is: if information or ideas are obtained from any source, that source must be acknowledged according to the appropriate convention in that discipline; and any direct quotation must be placed in quotation marks and the source cited immediately. Any failure to acknowledge adequately or to cite properly other sources in submitted work is plagiarism. Under examination conditions, material learnt by rote or close paraphrase will be expected to follow the usual rules of reference citation otherwise it will be considered as plagiarism. Departments should provide guidance on other appropriate use of references in examination conditions.

Plagiarism is considered to be an act of fraudulence and an offence against University discipline. Alleged plagiarism, at whatever stage of a student's studies, whether before or after graduation, will be investigated and dealt with appropriately by the University.

The University reserves the right to use plagiarism detection systems, which may be externally based, in the interests of improving academic standards when assessing student work.